Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Ingeniería en Computación



Scheduling guiado por tipo de proceso para FreeBSD

Autores:

- Bonino, Francisco Ignacio – 41.279.796 – franbonino82@gmail.com
- Daniele, Francisco – 40.964.017 – frandaniele1998@gmail.com

Director:

- Ing. Eschoyez, Maximiliano Andrés

Co–director:

- Ing. Papp, Nicolás

Córdoba, Julio de 2024







Dedicatorias

A nuestras familias, cuyo apoyo incondicional fue fundamental para recorrer este camino de principio a fin.



Agradecimientos

A los ingenieros Maximiliano Eschoyez y Nicolás Papp por la confianza depositada al darnos la oportunidad de realizar este trabajo bajo el marco del Proyecto Scheduler.

Al Laboratorio de Arquitectura de Computadoras por brindar asesoramiento teórico y práctico para la realización del presente trabajo.



Resumen

El sistema operativo se basa en un conjunto de programas que controlan los procesos de una computadora o sistema embebido. Su principal función es permitir a los programas de usuario acceder a los recursos del *hardware* utilizado. Dentro de sus tareas más importantes se encuentra la planificación, que consiste en el reparto de tiempo de procesador a los diferentes hilos y procesos que se encuentran en ejecución.

El *Proyecto Scheduler* es un proyecto que aborda una propuesta de planificación dinámica de procesos basándose en el planificador 4BSD del sistema operativo FreeBSD. Este proyecto está conformado por múltiples trabajos previos y contemporáneos al aquí presentado.

En este proyecto integrador se busca la actualización y unificación de los trabajos del *Proyecto Scheduler* a un mismo entorno de trabajo con las modificaciones que resulten pertinentes, y se explorará una prueba de concepto de planificación de procesos en ejecución basándose en las necesidades de los mismos. Para lograrlo, un mecanismo de pasaje de mensajes de espacio de usuario a espacio de *kernel* será usado para interactuar con una versión modificada de la red de Petri utilizada en los trabajos anteriores y así poder cambiar el comportamiento del *scheduler* de manera dinámica. Este enfoque permitirá ofrecer una alternativa a la asignación de los recursos de la CPU en entornos con carga de trabajo variable en un sistema multiprocesador simétrico.

A lo largo del proyecto se considerarán aspectos como la interoperabilidad de los módulos de *kernel* ya existentes y los desarrollados en este trabajo, las buenas prácticas de programación en un entorno de bajo nivel, y la escalabilidad del trabajo resultante.

Palabras clave

Sistemas Operativos, Redes de Petri, Planificador, Kernel.



Abstract

The operating system is based on a set of programs that control the processes of a computer or embedded system. Its main function is to allow user programs to access the hardware resources being used. Among its most important tasks is scheduling, which involves allocating processor time to the different threads and processes that are running.

The *Proyecto Scheduler* is a project that addresses a proposal for dynamic process scheduling based on the 4BSD scheduler of the FreeBSD operating system. This project consists of multiple previous and contemporary works.

The aim of this project is to update and unify the tasks of the *Proyecto Scheduler* into a single working environment with pertinent modifications. Additionally, a proof of concept for an alternative scheduling mechanism based on the needs of running processes will be explored. To achieve this, a mechanism for communication from user space to kernel space will be used to interact with a modified version of the Petri net used in previous works and thus dynamically change the scheduler's behavior. This approach will provide an alternative to CPU resource allocation in environments with variable workloads in a symmetric multiprocessor system.

Throughout the project, considerations will be given to aspects such as interoperability of existing kernel modules and those developed in this work, best programming practices in a low-level environment, and scalability of the resulting work.

Keywords

Operating Systems, Petri Nets, Scheduler, Kernel.



Índice de contenidos

1. Introducción....10
1.1 Motivación....10
1.2 Objetivos....10
1.3 Alcance....11
1.4 Modelo de desarrollo....11
2. Base teórica....12
2.1 Introducción....12
2.2 FreeBSD....12
2.3 Procesos e hilos....12
2.3.1 Estados de los procesos....13
2.4 Planificador....13
2.5 Red de Petri....14
2.5.1 Componentes de una red de Petri....14
2.5.2 Propiedades de una red de Petri....16
2.5.3 Evolución de una red de Petri....17
2.6 Ejecutables ELF....18
2.6.1 Estructura....18
3. Sinergia con trabajos previos....20
4. Desarrollo....21
4.1 Primera iteración....21
4.1.1 Objetivos....21
4.1.2 Implementación....21
4.1.2.1 Ambientación....21
4.1.2.2 Compilación del *kernel*....24
4.1.2.3 Módulos incorporados....24
4.1.3 Análisis de los resultados....25
4.1.4 Próximos pasos....26
4.2 Segunda iteración....27
4.2.1 Objetivos....27
4.2.2 Implementación....27
4.2.3 Análisis de los resultados....29
4.2.4 Próximos pasos....30
4.3 Tercera iteración....31
4.3.1 Objetivos....31
4.3.2 Implementación....31
4.3.3 Análisis de los resultados....39
4.3.4 Próximos pasos....39
4.4 Cuarta iteración....40
4.4.1 Objetivos....40
4.4.2 Implementación....40
4.4.3 Análisis de los resultados....43



4.4.4 Próximos pasos 45
4.5 Quinta iteración 46
4.5.1 Objetivos 46
4.5.2 Implementación 46
4.5.2.1 callback_load 47
4.5.2.2 callback_monopolization 49
4.5.3 Análisis de los resultados 50
4.5.4 Próximos pasos 51
4.6 Sexta iteración 52
4.6.1 Objetivos 52
4.6.2 Implementación 52
4.6.2.1 Escalabilidad 52
4.6.2.2 Macros 54
4.6.2.3 Estructuras 55
4.6.2.4 Estadísticas 55
4.6.2.5 Módulo toggle_active_cpu 56
4.6.2.6 Optimización general 58
4.6.3 Análisis de los resultados 58
4.6.4 Próximos pasos 58
4.7 Séptima iteración 59
4.7.1 Objetivos 59
4.7.2 Implementación 59
4.7.3 Análisis de los resultados 60
4.7.4 Próximos pasos 60
4.8 Octava iteración 61
4.8.1 Objetivos 61
4.8.2 Implementación 61
4.8.3 Análisis de los resultados 62
4.8.4 Próximos pasos 64
4.9 Novena iteración 65
4.9.1 Objetivos 65
4.9.2 Implementación 65
4.9.3 Análisis de los resultados 70
4.9.4 Próximos pasos 70
4.10 Décima iteración 71
4.10.1 Objetivos 71
4.10.2 Implementación 71
4.10.3 Estado final 71
4.10.3.1 Versiones de FreeBSD 71
4.10.3.2 Red de Petri: *Scheduler* 4BSD 72
4.10.3.3 *Metadata* en ejecutables ELF 73
4.10.3.4 *Scheduling* por tipo de proceso 74
4.10.4 Análisis de los resultados 74
4.10.5 Próximos pasos 75



5. Síntesis de los resultados obtenidos....76
5.1 Suspensión de núcleos....76
5.2 Monopolización de núcleos....76
5.3 Incorporación de *metadata* en *headers* de ejecutables ELF....76
5.4 Recopilación de estadísticas....77
5.5 Toma de decisiones....77
5.6 Corolario....77
6. Trabajos futuros....78
7. Conclusiones....80
8. Bibliografía....82
A. Anexo: Secuencias de disparos en la red de Petri....83
A.1 Transición THROW....83
A.2 Suspensión y THROW....86
A.3 Suspensión con nuevo THROW....87
A.4 Suspensión y THROW con red simplificada....89
A.4.1 Problema....89
A.4.2 Solución....91
B. Anexo: Configuración del entorno de trabajo resultante....92
B.1 Introducción....92
B.1.1 Vagrant....92
B.1.2 Ansible....92
B.2 Aprovisionamiento de VMs con el proyecto integrador....92



1. Introducción

El presente proyecto integrador forma parte del entorno de trabajo del *Proyecto Scheduler*, dirigido por los ingenieros Maximiliano Eschoyez y Nicolás Papp en colaboración con el Laboratorio de Arquitectura de Computadoras (LAC), el cual ha desarrollado múltiples trabajos teóricos y prácticos utilizando redes de Petri para el modelado de sistemas concurrentes.

1.1 Motivación

En el contexto del desarrollo del sistema operativo FreeBSD, la planificación de procesos es un aspecto crítico para garantizar un rendimiento óptimo y una utilización eficiente de los recursos del sistema. Los planificadores integrados (ULE y 4BSD) han demostrado ser eficaces en muchos escenarios, pero existen oportunidades para explorar alternativas de manera práctica que puedan refinar la administración de recursos en escenarios de específico interés.

Nuestra principal motivación radica en el aporte a este proyecto mediante la evaluación práctica de alternativas de *scheduling* que puedan significar un beneficio para el estudio teórico del tema y así contribuir al producto final que se persigue.

1.2 Objetivos

El trabajo a desarrollar aquí forma parte del mencionado *Proyecto Scheduler*, por lo que mantiene como objetivo principal proponer un modelo de planificador del sistema operativo capaz de prever la mejor forma de asignar y mantener disponibles los recursos del sistema logrando aportar información sobre los procesos desde el espacio de usuario a espacio de *kernel*.

A nivel secundario, los objetivos de este trabajo son:

- La integración de los trabajos previos y contemporáneos que conforman también el *Proyecto Scheduler* a un mismo entorno de trabajo.
- Actualizar dichos trabajos a la última versión posible del sistema operativo FreeBSD.
- Proponer un mecanismo de *scheduling* alternativo implementando una interfaz de comunicación entre los procesos y el *scheduler* para proveer información de ejecución en las cabeceras de los ejecutables ELF que permita al planificador de corto plazo tomar decisiones, como inhibir núcleos inactivos de la CPU y



balancear la carga de trabajo, y permitir la monopolización de núcleos cuando un proceso lo solicite.

1.3 Alcance

El presente proyecto integrador toma como base los trabajos realizados bajo el *Proyecto Scheduler* para integrar los mismos en un entorno de desarrollo integral, y así posteriormente proponer un mecanismo de *scheduling* que unifique las funcionalidades ofrecidas por los mismos.

Se hará una prueba de concepto del funcionamiento de la propuesta sin llegar a análisis teóricos y prácticos exhaustivos ni de *performance*, ya que se trata de un primer acercamiento a una alternativa de planificación que integre todos los trabajos en el *kernel* del sistema operativo.

La propuesta desarrollada quedará adaptada a la versión 15 del sistema operativo FreeBSD y se proveerán guías de configuración de máquinas virtuales con el entorno funcional de este proyecto para facilitar el comienzo de futuros trabajos que tomen el presente como base.

1.4 Modelo de desarrollo

El enfoque escogido para alcanzar los objetivos planteados se basa en el desarrollo iterativo incremental. Este modelo de desarrollo plantea comenzar con una implementación y avanzar a través de versiones intermedias presentadas en nuevas etapas, que agregan nuevas funcionalidades y correcciones en caso de ser necesario. Este proceso se repite hasta llegar a una versión final. Esta metodología permite adaptar el proyecto rápidamente a los cambios, ya sean positivos o negativos, y lograr entregas continuas.

Para el trabajo colaborativo se utilizará un repositorio *git* alojado en *GitHub* donde se tendrá todo el código desarrollado, notas, diagramas, documentación y los recursos que deriven del trabajo que se realice. Además, el mismo está asociado a una herramienta de seguimiento provista por *GitHub* llamada *GitHub Projects* que permite visualizar y realizar seguimiento a las tareas del proyecto.

- Repositorio: `https://github.com/frandaniele/pi_doc`
- Seguimiento: `https://github.com/users/frandaniele/projects/3`



2. Base teórica

Previo al desarrollo del proyecto integrador, se deben asentar las bases teóricas necesarias para poder comprender los conceptos y términos abarcados a lo largo del trabajo, junto con las herramientas utilizadas para el análisis y la toma de decisiones.

2.1 Introducción

Al trabajar con un planificador de corto plazo en un sistema operativo como FreeBSD, hay que contar con una herramienta analítica que permita modelar el comportamiento del mismo. Esto con el fin de poder tomar decisiones de diseño, detectar escenarios críticos a tener en cuenta en el desarrollo y para conseguir una mejor organización a la hora de transcribir el modelo a código de *kernel*, tarea que requiere un manejo cuidadoso para evitar escenarios indeseados. Una red de Petri es el instrumento ideal para este propósito.

A continuación se brindará una explicación y análisis de los términos, conceptos y herramientas abordados en este proyecto.

2.2 FreeBSD

Basado en UNIX[1], FreeBSD es un sistema operativo de código abierto de alto rendimiento, estabilidad y seguridad, principalmente orientado a servidores robustos. Como sistema operativo, su tarea principal es la de actuar como intermediario entre el *hardware* de un computador y el software que se ejecuta en él, mediante la gestión de recursos, procesos, dispositivos y red. Es responsable también de proveer seguridad al sistema y a los datos de acceso no autorizados, y de la organización de la información mediante un sistema de archivos.

2.3 Procesos e hilos

Un proceso representa un programa en ejecución junto con su espacio de memoria que contiene un mapeo sobre el código del mismo, y los recursos asociados. Múltiples programas pueden ejecutarse concurrentemente en una computadora.

1 *Sistema operativo modular, multiusuario y multitarea, altamente configurable, desarrollado en los años 90 en los laboratorios de AT&T. Es usado como base de muchos sistemas operativos modernos.*



Un hilo es la unidad más pequeña de ejecución que forma parte de un proceso. Dado que un proceso puede estar formado por más de un hilo, los hilos comparten el mismo espacio de memoria y recursos que su proceso padre. Los hilos pueden comunicarse de manera más rápida y eficiente que los procesos, pero también requieren una sincronización cuidadosa para evitar problemas de concurrencia, como condiciones de carrera. Así como muchos procesos pueden ejecutarse de manera concurrente, los hilos permiten la ejecución concurrente de tareas dentro de un programa y son útiles para mejorar el rendimiento en sistemas multinúcleo.

2.3.1 Estados de los procesos

Los posibles estados en los que puede estar un proceso, y sus significados, son los siguientes:

- **Nuevo:** Está siendo cargado en memoria, aún no está listo para ejecutar.
- **Listo:** Puede pasar al estado de ejecución. Todas las tareas del proceso están listas para ejecutarse pero se espera a que el procesador quede libre (hay otros procesos más prioritarios en ejecución).
- **En ejecución:** Se está ejecutando actualmente en el procesador.
- **Bloqueado:** Está esperando la respuesta de algún otro proceso para poder continuar con su ejecución.
- **Terminado:** La ejecución del proceso ha sido finalizada exitosamente, o el sistema operativo ha detectado un error fatal y ha forzado su terminación.

2.4 Planificador

Para lograr administrar eficientemente la asignación de recursos a procesos que buscan ejecutarse, el sistema operativo cuenta con un componente esencial de planificación –*scheduler* en inglés–, que se divide en tres categorías según su alcance: de corto plazo, de mediano plazo y de largo plazo.

Un planificador de corto plazo se encarga de administrar la asignación de recursos de la CPU a los procesos en ejecución, decidiendo qué proceso se ejecutará a continuación y por cuánto tiempo.

Un planificador de mediano plazo gestiona la eficiencia en el uso de la memoria principal, administrando los procesos que están en un estado de suspensión temporal, decidiendo cuándo cargar o sacar procesos de la memoria principal a una memoria secundaria.



Un planificador de largo plazo se encarga de determinar cuántos procesos pueden estar activos en el sistema de manera simultánea, manteniendo un equilibrio entre el rendimiento y la eficiencia del sistema.

En el proyecto integrador abordado, se trabaja con el planificador de corto plazo del sistema operativo FreeBSD. El *kernel* de FreeBSD cuenta con dos planificadores disponibles:

- **4BSD**: Tradicional planificador de FreeBSD, presente desde la creación del sistema BSD, antecesor a FreeBSD. Cuenta con soporte para sistemas multinúcleo y multihilo.
- **ULE**: Planificador desarrollado más recientemente de FreeBSD, diseñado desde un principio para ofrecer rendimiento predecible en sistemas multinúcleo y multihilo. En comparación a 4BSD, mejora la gestión de la concurrencia y proporciona tiempos de respuesta más consistentes, incrementando su complejidad.

El planificador de corto plazo utilizado para este proyecto es el 4BSD por soportar sistemas SMP y su simpleza en comparación al ULE.

2.5 Red de Petri

Definidas por Carl Adam Petri, una red de Petri es una herramienta muy útil que se usa para modelar una gran variedad de fenómenos de informática y control. Estas redes permiten modelar y visualizar comportamientos con paralelismo, concurrencia, sincronización e intercambio de recursos.

Dado que las redes de Petri son un tema muy extenso y complejo, se procederá a describir los atributos y propiedades abarcados en este proyecto integrador.

2.5.1 Componentes de una red de Petri

Al ser una herramienta gráfica de modelado, debemos saber identificar cada componente y comprender sus roles.

- **Plaza**: Representadas gráficamente como círculos, las plazas son un tipo de nodo que representa una condición o un estado dentro del sistema modelado. La presencia de algo en este estado o condición se representa mediante *tokens*. Un *token* es una unidad básica de información que representa la presencia de una entidad en cierto estado o el cumplimiento de una condición específica.



Representado mediante un punto dentro de una plaza, un *token* se utiliza para modelar el estado de un sistema y facilitar el seguimiento del flujo de información o recursos.

- **Transición**: Una transición es otro tipo de nodo, con el rol de detector de condiciones y transformador de estados. Se simboliza en una red de Petri como un rectángulo. Una transición se considera sensibilizada cuando se cumplen las condiciones necesarias para *dispararla*. Estas condiciones se detallarán más adelante. Una transición se considera instantánea cuando el *disparo* de la misma se efectúa de manera instantánea; es decir, el tiempo en el cual se realiza el cambio de estado de la red debido al disparo de la transición es cero. Una transición se considera automática cuando el *disparo* de la misma se efectúa obligatoriamente tan pronto como se cumplan las condiciones necesarias. Una transición se considera jerárquica cuando está compuesta por subtransiciones que deben dispararse en un orden estricto para que la transición de mayor jerarquía pueda dispararse.
- **Arco**: Un arco es un conector de nodos en una red de Petri, y tiene una dirección asociada, definiendo así un nodo de origen y un nodo de destino. Las únicas dos direcciones posibles de un arco son:
  - De una plaza a una transición.
  - De una transición a una plaza.

  La función de un arco es la de establecer las condiciones de sensibilizado de una transición mediante su peso. El peso es una propiedad que pueden tener algunos arcos, e indica cuántos *tokens* se necesitan para cumplir una condición de sensibilizado de una transición, o cuántos *tokens* se colocarán en una plaza, dependiendo de los nodos de origen y destino que estén unidos por el arco. De esta forma, si un arco A tiene una plaza P como nodo de origen, una transición T como nodo de destino, y un peso $M > 0$, entonces decimos que la plaza P debe tener *al menos* M *tokens* para que la transición T cumpla una condición de sensibilizado. Esta condición puede ser la única necesaria para efectuar el disparo, o puede ser parte de un conjunto de condiciones. Al dispararse la transición T, los M *tokens* de la plaza P serán consumidos.

  Si el mismo arco A de peso M tiene como nodo de origen una transición T y como nodo de destino una plaza P, entonces decimos que cuando la transición T se *dispare*, se añadirán M *tokens* a la plaza P.



Un arco directo en una red de Petri es un arco que tiene un nodo de origen, uno de destino, un peso (si no se especifica, el peso por defecto de un arco directo es 1), y efectúa la transición de *tokens* especificada en el ítem anterior. Se simboliza como una flecha que va del nodo de origen hacia el nodo de destino.

Un arco inhibidor es un arco A que tiene una plaza P como nodo de origen y una transición T como nodo de destino, pero que cumple la función opuesta a la de un arco directo: la condición de sensibilizado para la transición de origen T es que la plaza de destino P *no tenga ningún token*. Se simboliza como un arco que en el extremo del nodo de destino tiene un círculo en lugar de una punta de flecha. Los arcos inhibidores no tienen peso, ya que indican que la transición no puede dispararse si hay *al menos un token* en la plaza de donde sale el arco.

2.5.2 Propiedades de una red de Petri

- **Marcado**: Este concepto es sólo aplicable a plazas individualmente y a redes de Petri en su totalidad. Para una plaza, el marcado es un número natural finito que indica la cantidad de *tokens* que posee la misma. Para una red de Petri, el marcado es un vector de dimensión $1 \times N$, siendo N el número de plazas de la red, donde cada elemento indica el marcado de la plaza que representa.

- **Acotación**: Una red de Petri se considera acotada cuando está limitada la cantidad máxima de *tokens* posibles en cada plaza. Esta propiedad es útil para simular sistemas con recursos o condiciones de cantidad finita y limitada, donde se busca analizar la capacidad y el rendimiento del sistema.

- **Seguridad**: Una red de Petri se considera segura cuando la cantidad de *tokens* que puede tener cada plaza es, como máximo, uno. Dado que esto ayuda a garantizar que no haya conflictos en la red, esta propiedad es útil para modelar sistemas con recursos compartidos.

- **Interbloqueo**: También conocido como *deadlock*, un interbloqueo en una red de Petri es un estado de la red en el que no se pueden disparar más transiciones, o las transiciones sensibilizadas representan un bucle infinito que impide el correcto avance de la red.



- **Pureza**: Una red de Petri se considera pura cuando ninguna transición contiene una plaza que sea simultáneamente de entrada y de salida.

2.5.3 Evolución de una red de Petri

Con los componentes y propiedades listados anteriormente, estamos en condiciones de comenzar a analizar la evolución de una red de Petri.

- **Matriz de incidencia**: La matriz de incidencia de una red de Petri es una matriz que representa las relaciones entre los nodos de la red y los arcos que los conectan. Esta matriz es de dimensión $m \times n$, donde m (filas) es la cantidad de plazas y n (columnas) es la cantidad de transiciones de la red. Cada elemento de la matriz indica si existe un arco que conecta los nodos y si el arco es *de entrada* o *de salida*. Los valores que pueden tomar estos elementos son los siguientes:
  - 0: No hay conexión entre los nodos.
  - 1: Existe un arco *de entrada* que conecta los nodos. Esto significa que el nodo de origen es una plaza y el nodo de destino es una transición.
  - -1: Existe un arco *de salida* que conecta los nodos. Esto significa que el nodo de origen es una transición y el nodo de destino es una plaza.
- **Matriz de inhibición**: La matriz de inhibición de una red de Petri es una matriz que representa las relaciones de inhibición entre nodos de la red. La estructura de la matriz y el orden de sus filas y columnas es igual a la matriz de incidencia, pero en este caso los valores que puede tomar cada elemento son los siguientes:
  - 0: No hay relación de inhibición entre nodos.
  - 1: Existe una relación de inhibición entre nodos.
- **Vector de disparo**: Todo disparo de una transición sensibilizada es atómico; es decir, indivisible. A la hora de definir qué transición se va a disparar, se especifica un vector de disparo de dimensión $1 \times m$, siendo m el número de transiciones de la red. Los valores de los elementos de este vector pueden ser los siguientes:
  - 0: La transición no será disparada.
  - 1: La transición será disparada.



- **Ecuación de estado**: La ecuación de estado de una red de Petri es una representación matemática que describe el estado próximo de la red en términos del marcado actual de la misma y su matriz de incidencia, de la siguiente forma:

$$\mu_{j+1} = \mu_j + I\sigma$$

Siendo:

  - μ_j: Vector de marcado actual de la red.
  - μ_{j+1}: Vector de marcado del estado próximo de la red.
  - I: Matriz de incidencia de la red.
  - σ: Vector de disparo.

2.6 Ejecutables ELF

Del inglés *Extensible Linkable Format*, este formato es un estándar común multiplataforma para archivos ejecutables presente en sistemas operativos UNIX que se caracteriza por su flexibilidad, extensibilidad y por admitir cualquier tipo de CPU y arquitectura de instrucciones.

2.6.1 Estructura

La estructura básica de un ejecutable ELF consta de las siguientes partes:

- **Header**: Información general sobre el ejecutable. Aquí se indica, por ejemplo, la arquitectura de sistema requerida por el archivo, si las direcciones a utilizar por el ejecutable son de 32 o 64 bits, si su formato es *little–endian* o *big–endian*. Además, contiene las posiciones de las tablas de *headers* de programas y de los *headers* de sección. Estas últimas son de nuestro interés y suelen coincidir en todos los sistemas:
- `.text`: Instrucciones ejecutables del programa.
- `.rodata`: Datos de sólo lectura (*read–only data*) sobre el programa.
- `.data`: Datos de programa inicializados que se necesitarán en memoria para la ejecución del programa.
- `.debug`: Información para depuración simbólica.
- `.dynamic`: Información de enlace dinámico.
- `.init`: Instrucciones que se ejecutarán durante la inicialización del proceso.



- `.fini`: Instrucciones que se ejecutarán durante la terminación del proceso.

De especial interés para este proyecto integrador será la sección `.metadata` diseñada por Cancinos – González [2], ya que la misma permite el traspaso de información personalizada desde espacio de usuario a espacio de *kernel*.



3. Sinergia con trabajos previos

Este proyecto integrador cuenta con la colaboración de otros autores en forma de integración de trabajos ya realizados, que ayudaron a incorporar las herramientas y funcionalidades listadas a continuación:

- Modelo base del planificador de corto plazo 4BSD mediante red de Petri [1].
- Comunicación entre aplicaciones de usuario y el *kernel* del sistema operativo mediante archivos en formato ELF [2].
- Monopolización y cambio de estado de actividad de núcleos de procesamiento de la CPU [3].



4. Desarrollo

Al adoptar la modalidad de desarrollo iterativo incremental, cada iteración presentará los objetivos propuestos para la misma, el desarrollo teórico de los objetivos en caso de ser necesario, su implementación, un análisis de los resultados obtenidos, y finalmente las consideraciones para la iteración siguiente.

Una condición mandatoria para el desarrollo de este proyecto fue la de trabajar sobre la versión más reciente del sistema operativo FreeBSD, que puede obtenerse directamente desde el repositorio oficial del código fuente, o desde la página oficial. Teniendo en cuenta que este proyecto comenzó en Mayo de 2023, se tomó como más reciente la versión 13.2, sabiendo que para finales del mes de Octubre de 2023 está programado el *release* de la versión 14.0, por lo que todo archivo afectado deberá ser tenido en cuenta para una actualización del proyecto a dicha versión una vez sea publicada.

4.1 Primera iteración

4.1.1 Objetivos

Habiendo mencionado la colaboración con trabajos desarrollados por otros autores, se debe tener en cuenta que los archivos pertinentes del sistema operativo FreeBSD han sufrido cambios a la fecha, por lo que el código fuente de estos trabajos debe ser actualizado a la versión del sistema operativo que se va a utilizar para este proyecto en esta iteración (versión 13.2).

Es por esto que el objetivo de esta primera iteración es la adaptación de los trabajos referenciados a la versión más reciente de FreeBSD a la fecha.

4.1.2 Implementación

4.1.2.1 Ambientación

El primer paso fue ambientarse en este sistema operativo, más precisamente en lo relacionado al *Proyecto Scheduler*. Por esto es que inicialmente se dedicó mucho tiempo a leer *papers* y documentación sobre FreeBSD, sobre desarrollo de *kernel* y módulos de *kernel*, compilación de *kernels* personalizados, *debugging* y *scheduling*.

Como primer paso, se estudió la red de Petri tomada como base [1], llegando a identificar las siguientes secciones claves:

- La primera sección muestra los posibles estados en los que puede estar un núcleo de la CPU:

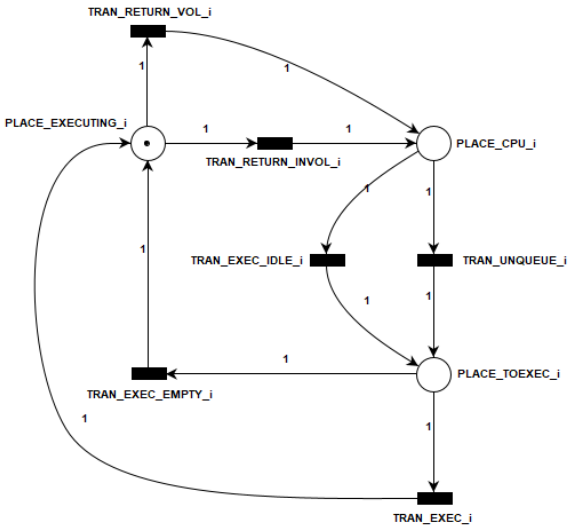


Figura 1: Estados posibles de un núcleo de una CPU

- La segunda sección, que involucra a la primera, tiene en cuenta el encolado global:

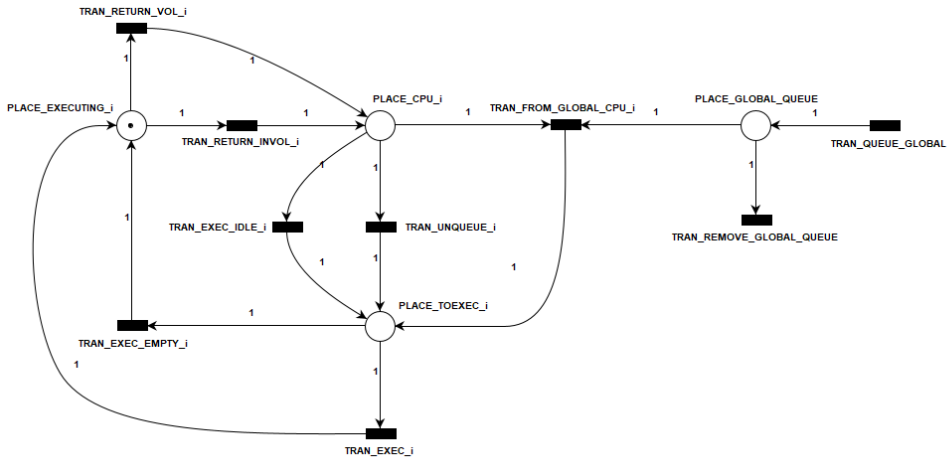


Figura 2: Encolado global en núcleos de una CPU

- La tercera sección introduce a la primera el mecanismo de encolado por núcleo:

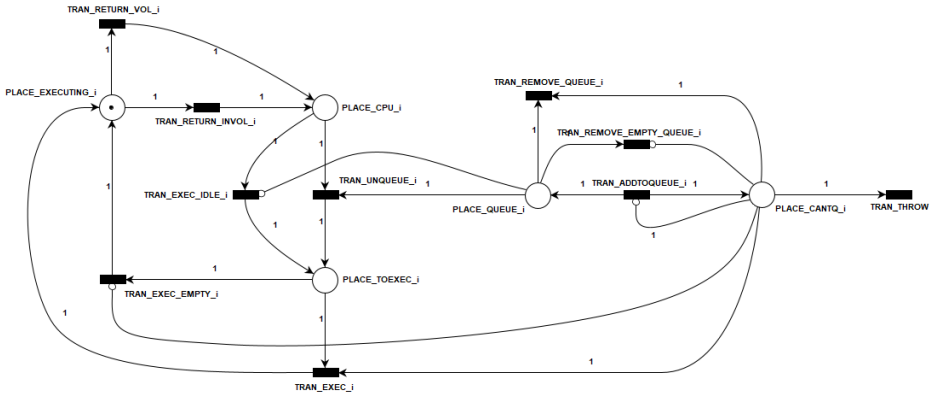


Figura 3: Encolado por núcleo en una CPU

Finalmente, una vista de la red completa tomada de base para un sistema de una CPU con 4 núcleos se puede ver a continuación:

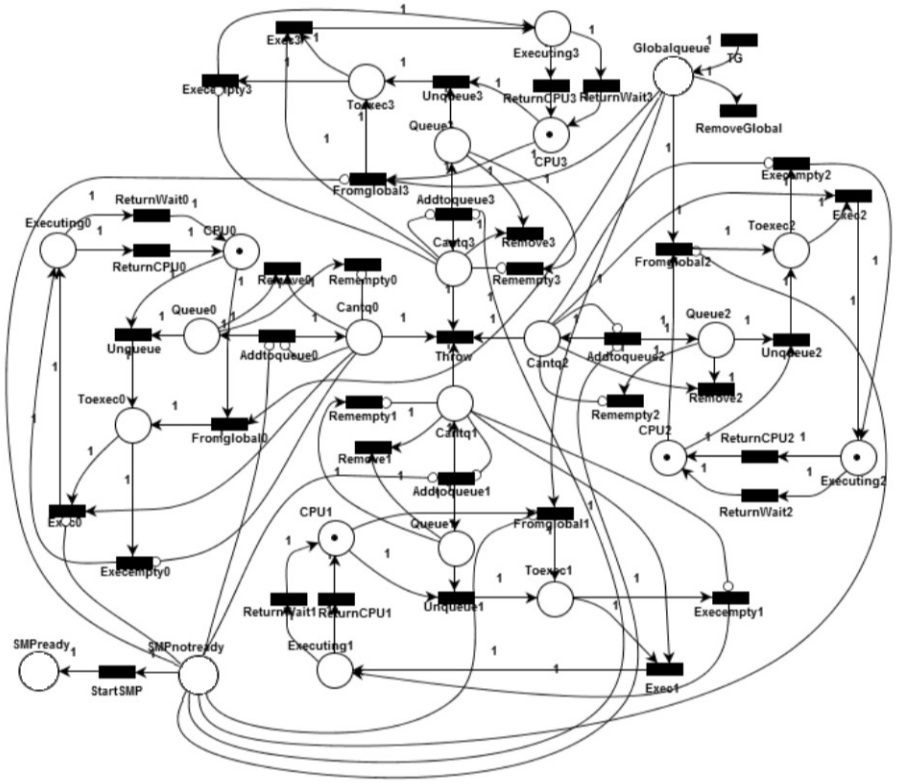


Figura 4: Red base completa para un sistema de una CPU con 4 núcleos



Llegado el punto en que logramos familiarizarnos con el sistema operativo y la representación de su *scheduler* mediante redes de Petri, se procedió a preparar máquinas virtuales con la versión 13.2 de FreeBSD, configurarlas y empezar a estudiar el código de los proyectos integradores complementarios para adaptarlos a esta nueva versión del sistema operativo.

4.1.2.2 Compilación del *kernel*

Al tratarse de desarrollo a nivel de *kernel*, para ver reflejados en el sistema los cambios que uno realiza sobre el código se debe recompilar el *kernel* con los archivos nuevos/modificados. Los archivos compilados por el *kernel* se encuentran dentro del *source tree* que por defecto su *path* es `/usr/src/sys/`. Este proceso puede llevar mucho tiempo, por lo que es importante conocerlo a fondo y optimizarlo lo mayor posible.

El primer paso fue reducir el *kernel* lo más posible, modificando la configuración por defecto apagando todos los módulos que no se necesitan para las máquinas virtuales. Una vez configurado y compilado este nuevo *kernel* minimalista, se empezó a trabajar sobre la actualización de los módulos de los proyectos integradores anteriormente mencionados. Para ello se agregaron sus archivos fuente a la configuración de este *kernel* para tenerlos en cuenta en la compilación.

4.1.2.3 Módulos incorporados

El módulo ***Metadata ELF FreeBSD*** comprende los siguientes archivos:

- `kern/metadata_elf_reader.c`
- `sys/metadata_elf_reader.h`
- `sys/metadata_payloads.h`

Y modifica los siguientes:

- `kern/imgact_elf.c`
- `kern/kern_exec.c`
- `sys/proc.h`



Este trabajo fue originalmente desarrollado sobre FreeBSD 12.3, por lo que las modificaciones realizadas por los desarrolladores de FreeBSD para la versión 13.2 no trajeron grandes dificultades para adaptar este modulo, ya que no afectaron la funcionalidad de los sectores del código donde se implementa. Es por esto que sólo se agregaron las líneas y funciones de código pertinentes, se incluyeron en el *kernel* y se recompiló el sistema.

Una vez obtenido este nuevo *kernel*, se probó que todo funcione correctamente utilizando los *plugins CLang* y *GCC* para insertar metadata en los ejecutables ELF y leer la misma en espacio de *kernel*.

El módulo ***Petri net scheduler*** comprende los siguientes archivos:

- `kern/petri_global_net.c`
- `sys/petri_global_net.h`
- `kern/sched_petri.c`
- `sys/sched_petri.h`

Y modifica los siguientes:

- `kern/sched_4bsd.c`
- `kern/kern_thread.c`
- `sys/proc.h`

Este proyecto se desarrolló en FreeBSD 11, por lo que debido a los cambios entre versiones en estos archivos, su adaptación a la nueva versión del *kernel* tuvo mayor dificultad ya que hubo algunas modificaciones en el comportamiento de funciones que forman parte del *scheduler* modelado con la red de Petri.

La mayor parte del código desarrollado por quienes realizaron este trabajo se agregó en sus respectivos archivos y funciones sin dificultad, a excepción del código en la función `sched_switch` que se encarga de expulsar un hilo en ejecución y seleccionar uno nuevo para reemplazarlo.

Una vez compilado el *kernel* con las modificaciones mencionadas, se empezó a utilizar el nuevo *scheduler* y a hacer pruebas para evaluar su comportamiento.

4.1.3 Análisis de los resultados

Luego del proceso de actualización del código de los proyectos integradores a la versión 13.2 de FreeBSD se comenzó a utilizar el sistema operativo con el *kernel* compilado con estos archivos y se logró el cometido de la iteración.



A pesar de esto, se observó que el código de selección de núcleos de la CPU para la ejecución de un hilo en el caso del *scheduler* modelado con la red de Petri modificaba el comportamiento original del *scheduler* 4BSD, ignorando la afinidad de los hilos (*flags* `td_pinned`, `TDF_BOUND` y `TSF_AFFINITY`). Por esto es que se buscó volver al esquema original de selección de núcleos teniendo en cuenta la afinidad y tomando las decisiones en base al modelo de la red de Petri. Aquí notamos *kernel panics*[2] constantemente, por lo que decidimos que nuestro próximo paso sería solucionar este problema.

4.1.4 Próximos pasos

Teniendo los trabajos actualizados a la versión 13.2 de FreeBSD, y habiendo notado la necesidad de modificar la red de Petri por el inconveniente detallado, se decidió que el próximo paso será ahondar en las modificaciones necesarias tanto en el modelado del *scheduler* como en el código para salvar el problema.

2 *Situación en la que el kernel del sistema operativo encuentra un error crítico del cual no puede recuperarse de manera segura. Esto generalmente resulta en la detención completa del sistema para evitar posibles daños adicionales.*



4.2 Segunda iteración

4.2.1 Objetivos

Se estableció como objetivo para esta segunda iteración la modificación de la red de Petri que modela el comportamiento del *scheduler* 4BSD para solucionar el problema relacionado con el escenario donde el nuevo modelo de *scheduler* ignoraba la afinidad de los procesos a algún núcleo de la CPU.

4.2.2 Implementación

Gran parte de esta iteración se basó en depurar el sistema para encontrar la causa del problema de los constantes *kernel panics* observados. Esta fue una tarea tediosa, ya que cada vez que se desea probar algún cambio se debe recompilar el *kernel*. Si este cambio produce un *kernel panic*, el sistema se debe reiniciar y el nuevo *kernel* queda inutilizable. Es por esto que se generó un *kernel* sano para utilizar cada vez que el sistema se rompa, y se duplicó la imagen de FreeBSD para levantar otra máquina virtual y poder depurar el *kernel* de manera *online*.

Luego de numerosas pruebas, se detectó que los *kernel panics* se daban cuando el modelo original asignaba a un hilo la cola del último núcleo de la CPU en donde se ejecutó, pero el modelo del *scheduler* de la red de Petri encontraba que esa cola no estaba disponible y le asignaba otra. Investigando acerca de la afinidad de hilos a núcleos de la CPU se encontró en la documentación de la arquitectura SMP[3] de FreeBSD que el *scheduler* debe garantizar que un hilo *pinned*[4] sea ejecutado únicamente en la CPU que se encontraba ejecutándolo al momento que la llamada a `sched_pin` fue realizada.

Llegado el punto en que se comprendió el por qué de estos fallos, se comenzó a analizar cómo salvarlos. Se entendió que en el caso de que un hilo esté asociado a un núcleo de la CPU, se debe ignorar si la transición para encolar en ese núcleo está sensibilizada. Para esto, se modificó la red de Petri agregando una nueva transición

3 *Symmetric multiprocessing. Diseño en el que dos o más procesadores idénticos comparten la misma memoria principal y están conectados a través de un bus de sistema para distribuirse la carga de trabajo y así mejorar el rendimiento y la capacidad de respuesta.*

4 *Hilo de ejecución cuyo estado está anclado o fijado a un núcleo específico de la CPU. Esto significa que el sistema operativo asigna permanentemente el hilo a un núcleo particular y no lo mueve entre núcleos.*

`TRAN_ADDTOQUEUE_PINNED`, la cual es similar a `TRAN_ADDTOQUEUE` con la diferencia de que la nueva no está inhibida cuando hay otros hilos en la cola del núcleo.

En este punto cabe resaltar que se tomó la decisión de modificar la red de Petri de tal manera que conserve su propiedad de pureza, en pos de preservar la simplicidad y claridad de la red, facilitando su análisis y evitando ciclos de retroalimentación que podrían causar comportamientos no deseados o difíciles de prever.

Los cambios se reflejan en la red de Petri como se muestra a continuación:

- Encolado:

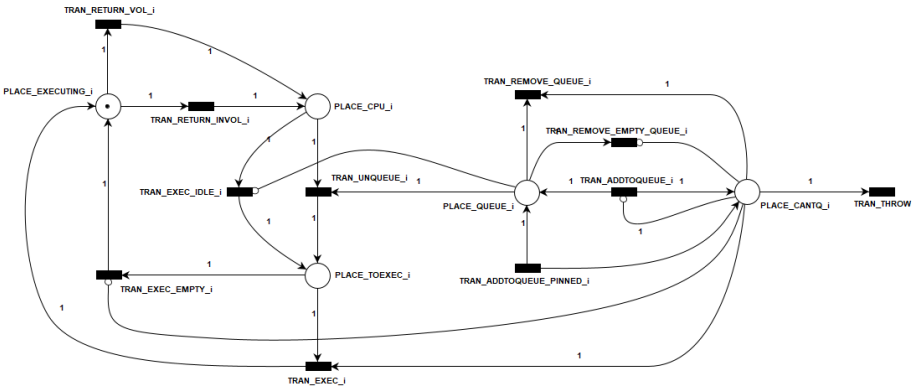


Figura 5: Red de Petri actualizada para el encolado en un núcleo de una CPU

- Red completa:

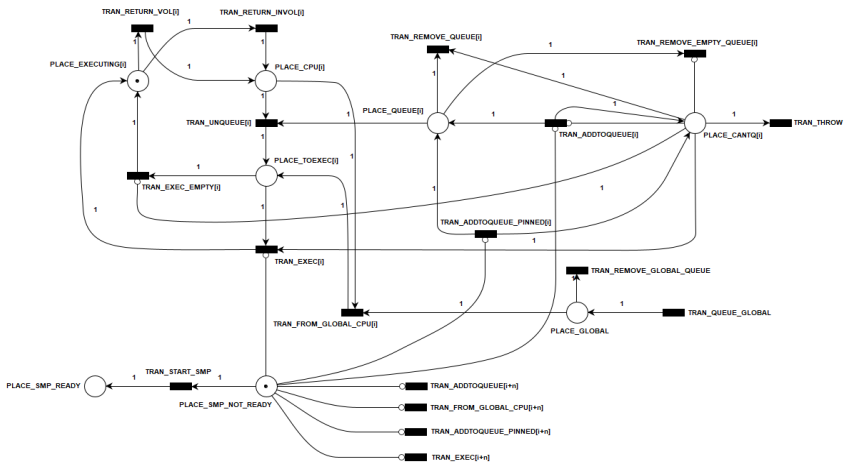


Figura 6: Red de Petri completa para un núcleo de una CPU



Una vez modificada la red, se incorporaron los cambios al código referentes a la definición de la red (matrices, números y nombres de transiciones en `petri_global_net.c`) y se tuvo que modificar el archivo `sched_4bsd.c` para el caso que de un hilo `td_pinned` pueda encolarse en la cola de su última CPU. La siguiente sección es la porción de código original que se encarga de este propósito:

```
1  if (td->td_pinned != 0) {
2    cpu = td->td_lastcpu;
3  } else if (td->td_flags & TDF_BOUND) {
4    KASSERT(SKE_RUNQ_PCPU(ts), ("sched_add: bound td_sched not on cpu runq"));
5    cpu = ts->ts_runq - &runq_pcpu[0];
6  } else {
7    cpu = sched_pickcpu(td);
8  }
9
10 int transition_number = TRANSITION(cpu, TRAN_ADDTOQUEUE);
```

Y la siguiente sección es la modificada para el escenario desarrollado en esta iteración:

```
1  boundcpu = ts->ts_runq - &runq_pcpu[0];
2  if (td->td_pinned != 0) {
3    cpu = td->td_lastcpu;
4  } else if (
5    td->td_flags & TDF_BOUND
6    && transition_is_sensitized(TRANSITION(boundcpu, TRAN_ADDTOQUEUE))
7  ) {
8    KASSERT(SKE_RUNQ_PCPU(ts), ("sched_add: bound td_sched not on cpu runq"));
9    cpu = boundcpu;
10 } else {
11   cpu = sched_pickcpu(td);
12 }
13
14 int transition_number = TRANSITION(cpu, TRAN_ADDTOQUEUE);
15
16 if (td->td_pinned != 0 && !transition_is_sensitized(transition_number)) {
17     transition_number = TRANSITION(cpu, TRAN_ADDTOQUEUE_PINNED);
18 }
```

4.2.3 Análisis de los resultados

Al modificar la red de Petri que modela el *scheduler*, logramos salvar el caso en el que un proceso está asociado a un núcleo de la CPU, consiguiendo así dejar de obtener los *kernel panics* que impedían escalar este proyecto a esos escenarios de ejecución de procesos.

Para corroborar que esta modificación no significa un problema en la estructura de la red de Petri, se analizaron las propiedades de la misma



comparándolas con el análisis final de proyecto integrador utilizado como referencia, y obtenemos los mismos resultados:

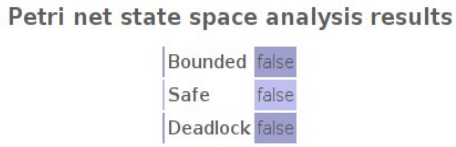


Figura 7: Análisis de la red de Petri modificada

Vemos así que conservamos una red no acotada, (debido a plazas de encolado), no segura y libre de *deadlocks*.

4.2.4 Próximos pasos

Una vez obtenida la adaptación del *scheduler* con el modelo de red de Petri corregido e incorporado a la versión 13.2 de FreeBSD, se decidió que el objetivo de la siguiente iteración será investigar sobre estadísticas de uso de núcleos del sistema, tratar de recolectar esta información y finalmente ponerla a disposición para su análisis y uso.



4.3 Tercera iteración

Teniendo adaptados exitosamente los trabajos a la versión 13.2 del sistema operativo FreeBSD, y teniendo el modelado del *scheduler* corregido para contemplar el caso en el que se tengan procesos cuya ejecución está asociada a un núcleo específico de la CPU, se llegó al momento de definir la utilidad práctica para darle a la combinación de estas herramientas.

Se decidió por abordar la implementación de un mecanismo de *scheduling* de procesos en un sistema monoprocesador multinúcleo mediante información recibida por *metadata*, con monitorización en tiempo real de la carga de cada uno de los núcleos de la CPU, implementando un mecanismo de conmutación dinámica del estado de actividad de cada núcleo controlado por el *scheduler* de corto plazo 4BSD modelado mediante la red de Petri resultante de la iteración anterior.

4.3.1 Objetivos

Para esta iteración, lo que se buscó fue establecer una primera aproximación al recopilado y procesamiento de estadísticas de uso de los núcleos de la CPU mediante un módulo de *kernel*.

4.3.2 Implementación

Se estudió mediante la herramienta `ktrace` las distintas llamadas a sistema que realizan los programas que muestran estadísticas como `ps` y `top`. Así se descubrió que hacen uso de la función `sysctl` para acceder a `kern.cp_time` y `vm.loadavg`.

Se exploró el *código fuente* de la versión 13 de FreeBSD para encontrar las estructuras, funciones y variables que utiliza el sistema operativo para almacenar y compartir estadísticas. También se investigaron las variables que tienen las estructuras de hilos y procesos referidas al uso de la CPU.

Una vez obtenida la base teórica, se decidió comenzar a desarrollar un módulo de *kernel* para recopilar estas estadísticas y tomar decisiones en base a ellas, pero inmediatamente surgió una cuestión fundamental de discutir: ¿qué estadísticas de uso recopilar?

En FreeBSD es posible obtener estadísticas detalladas sobre el uso de los núcleos de la CPU desglosadas para los distintos modos de ejecución de los procesos. Estos modos incluyen `USER` (usuario), `SYS` (sistema), `INTR` (interrupciones), `NICE` (prioridad ajustada) y `IDLE` (inactividad). Estas métricas proporcionan una visión



integral del rendimiento del sistema, permitiendo un análisis preciso de cómo se distribuye la carga de trabajo entre los diferentes modos de ejecución, lo que resulta fundamental para optimizar el desempeño y la eficiencia del sistema. Como primer acercamiento, optamos por medir la carga causada por los procesos de usuario.

La primera opción para lograr esto se basaba en ignorar los hilos de procesos de *kernel*. Para esto existe la estructura `pcpu` que almacena información específica de cada CPU del sistema y permite un seguimiento de la actividad de sus núcleos. Esta estructura cuenta con un campo `pc_curthread`, otra estructura que representa al hilo que está actualmente en ejecución en la CPU en cuestión. Esta estructura a su vez tiene un campo `td_proc` que funciona como puntero al proceso padre de este hilo. Este campo posee un campo señalizador de múltiples características de este proceso, llamado `p_flag`. Una de las posibles *flags* que este campo puede tener activada es `P_KPROC`, la cual indica si es un proceso de *kernel*. Si bien eso podría funcionar, nuestro propósito era contar con los hilos de procesos en modo de ejecución `USER` únicamente, por lo que habría que hacer una detección similar para el resto de modos de ejecución a ignorar. Una segunda posibilidad tuvo que ser tenida en cuenta.

Nuevamente entró en juego la estructura `proc`, la cual contiene también un campo `p_ucred` (una estructura `ucred`) que engloba los datos de las credenciales del proceso. Aquí se puede encontrar el campo `cr_uid`, el cual representa el número de identificación del usuario (*user* ID) que lanzó la ejecución del proceso. En sistemas UNIX, el *root user* (asociado con procesos de *kernel*) típicamente se representa con el ID 0. Notamos rápidamente que la documentación de este campo indica que no se debe leer su valor directamente, sino que se debe acceder al mismo mediante la interfaz `priv(9)` para verificación de privilegios de procesos mediante métodos que validan si un hilo tiene un nivel de privilegio pasado por parámetro. Esto nos hizo descartar esta opción automáticamente porque nuestro objetivo era mantener el *overhead* de procesamiento al mínimo en estos módulos, y si debemos acceder periódicamente a evaluar los privilegios de los procesos en ejecución mediante esta interfaz, tendríamos una penalización muy alta. Una vez más había que buscar otro camino.

En el *kernel* de FreeBSD existe la función `pcpu_find` que se utiliza para buscar y recuperar la estructura `pcpu` asociada con un ID de CPU específico (en este caso, núcleos de la CPU). Por otro lado, existe una *macro*[5] llamada `CPU_FOREACH` que se utiliza para facilitar la iteración entre las CPUs del sistema. Finalmente, en la estructura `pcpu` se tiene el campo `pc_cp_time`, que es un arreglo que mantiene un

5 *Definición de fragmentos de código que serán sustituidos por el preprocesador antes de la compilación del programa.*



registro del tiempo de CPU consumido por diferentes modos de ejecución en esa CPU específica, teniendo como índices las definiciones de modos de ejecución en el archivo `/usr/src/sys/sys/resource.h` (`CP_USER`, `CP_NICE`, `CP_SYS`, `CP_INTR` y `CP_IDLE`). Así, combinando estas herramientas, dimos con la forma más óptima de obtener las estadísticas de uso para cada modo de ejecución: basta con iterar cada núcleo de la CPU y obtener las estadísticas de uso en modo usuario directamente desde este arreglo con el índice correspondiente.

Se comenzó con un módulo de *kernel* básico para introducirse en el desarrollo de los mismos, y de a poco se le incorporaron utilidades hasta dividirlo en 2 módulos separados con tareas bien diferenciadas: uno para la recopilación de estadísticas referidas exclusivamente a cada núcleo de la CPU (`cpu_stats.c`) y el otro para la recopilación de estadísticas de los hilos y procesos (`thread_stats.c`). Cada uno de estos módulos posee un archivo *makefile* para su compilación y para ofrecer una fácil carga y descarga de los mismos en el *kernel* del sistema operativo.

El primer acercamiento fue con el módulo `cpu_stats`, el cual una vez cargado recopila las estadísticas de uso de cada núcleo de la CPU en formato de *ticks* de manera periódica, y el *string* resultante es escrito en un archivo de *log* que puede ser luego leído. Este archivo de *log* se sitúa en `/var/log/stats/cpus.log`. Para una mejor organización, se creó una librería compartida `modlib` para incluir definiciones, funciones, variables y estructuras comunes a ambos módulos. En esta primera oportunidad la información se escribía de manera muy verbosa, como se muestra en el ejemplo a continuación:

```
CPU 0
     USER
               Total: 20s | 0 % | 2608 ticks
               Delta: 3s | 38 % | 494 ticks
     NICE
               Total: 0s | 0 % | 0 ticks
               Delta: 0s | 0 % | 0 ticks
     SYS
               Total: 1s | 0 % | 192 ticks
               Delta: 0s | 0 % | 4 ticks
     INTR
               Total: 1s | 0 % | 137 ticks
               Delta: 0s | 0 % | 3 ticks
     IDLE
               Total: 2686s | 99 % | 341188 ticks
               Delta: 6s | 60 % | 783 ticks
TOTAL: 2709s (344125 ticks)
TOTAL 0 2709 344125 0
DELTA 0 10 1284 39
```



En este formato se muestra el tiempo en segundos, el porcentaje de uso y los *ticks* de cada modo, junto con los totales y sus *deltas* entre mediciones para evaluar qué tanto cambiaron las estadísticas.

Si bien los archivos de *log* eran muy completos y útiles, su tamaño crecía rápidamente al tener tanto detalle y esto no sólo consumía más tiempo de cálculo para los porcentajes y segundos y la escritura, sino que además dificultaba la búsqueda de información dentro del archivo a nivel de *kernel*, por lo que se decidió darle un nuevo formato menos verboso, reducido en cuanto a preservación de las estadísticas en formato de *ticks* únicamente, pero que igualmente sea intuitivo.

Llegamos a la siguiente estructura de *log* para cada núcleo de la CPU:

```
CPU_0_ticks_USER : 2608
CPU_0_ticks_NICE : 0
CPU_0_ticks_SYS : 192
CPU_0_ticks_INTR : 137
CPU_0_ticks_IDLE : 341188
CPU_0_ticks_TOTAL : 344125

CPU_0_DELTA_USER : 494
CPU_0_DELTA_NICE : 0
CPU_0_DELTA_SYS : 4
CPU_0_DELTA_INTR : 3
CPU_0_DELTA_IDLE : 783
CPU_0_DELTA_TOTAL : 1284
```

Con este nuevo formato podríamos acceder fácilmente a los valores de cada estadística por medio de una simple *regex*[6]:

```
CPU_$1_$2_$3[ \\t]*:[ \\t]*([0-9]+)(\\n|\\r)+?
```

En esta expresión regular basta con reemplazar los campos `$1` `$2` y `$3` por el número de núcleo de la CPU, el tipo de dato (`ticks` o `DELTA`) y el modo de ejecución del cual se quiere obtener las estadísticas, respectivamente, y automáticamente obtendremos el valor numérico correspondiente.

La expresión regular funciona, pero surgieron inconvenientes que impidieron su implementación. En primera instancia, la librería para manipulación de datos con expresiones regulares de GNU (`regex.h`) no está disponible a nivel de *kernel*, ya que estas operaciones pueden sobrecargar el sistema innecesariamente y pueden contener potenciales brechas de seguridad que comprometan al sistema operativo. Por otro

6 *Expresión regular (regular expression). Herramienta de patrones definidos utilizados para la coincidencia y manipulación eficiente de cadenas de texto dentro de un conjunto más grande de datos.*



lado, aunque la librería estuviese disponible, dimos con que el *parsing* de un archivo de texto puede ser algo muy costoso en tiempo y recursos, y la idea de un módulo de *kernel* es la de operar de manera rápida y ligera para causar el menor *overhead* posible en el rendimiento del sistema operativo. Fue por esto que se decidió adoptar una solución temporal que consistía en abrir los archivos de *log* desde el módulo de *kernel* que se encargaría de la lectura de las mismas.

Logramos definir las estadísticas de uso a recopilar y logramos almacenarlas y leerlas correctamente, y fue en este punto donde surgió la duda acerca de la frecuencia de muestreo de esta información. Una frecuencia muy alta podía suponer un *overhead* significativo para un módulo de *kernel*, resultando en la ralentización del sistema, y una frecuencia muy baja podía llevarnos a perder información importante de uso entre muestras. Al mismo tiempo debíamos tener en consideración el hecho de no trabajar con una frecuencia de muestreo constante, ya que si existe en el sistema algún proceso con ejecución de forma periódica que no coincida con la de muestreo, no podríamos recopilar información de su comportamiento.

Los comandos de referencia `top` y `htop` toman muestras de las estadísticas del sistema cada 3 y 2 segundos por defecto, respectivamente. Estos intervalos de tiempo son razonables para programas de monitorización general del sistema, pero para un *scheduler* pueden resultar intervalos muy grandes, ya que una CPU moderna puede ejecutar varios millones de instrucciones por segundo. Por este motivo decidimos mantenernos en un período de muestreo menor al segundo (400 a 600 milisegundos), controlado por un *timer* que marcará un período de tiempo elegido de manera aleatoria en dicho intervalo y funcionará como disparador para el siguiente muestreo, el cual tomará otro período de tiempo aleatorio en el mismo rango. Para esto definimos la siguiente función que calcula la frecuencia correspondiente al período elegido:

```
1  int obtain_random_freq(void) {
2    int random_interval_ms;
3    int freq;
4    int min_ms = 400;
5    int max_ms = 600;
6
7    random_interval_ms = min_ms + (random() % (max_ms - min_ms + 1));
8    freq = (random_interval_ms * hz)/1000;
9
10   return freq;
11 }
```



Y podemos utilizarla de la siguiente manera:

```
1  static void timer_callback(void *arg) {
2    read_cpu_stats();
3    callout_reset(&timer, obtain_random_freq(), timer_callback, NULL);
4  }
```

Hubo que adoptar también un criterio para clasificar la carga del sistema. Para esto se partió de mediciones sin ninguna carga adicional, obteniendo unas estadísticas de un 75% para procesos `IDLE` y un 25% para procesos `SYS` como puede apreciar a continuación:

```
last pid:   866;  load averages:  0.98,  0.57,  0.25    up 0+00:04:02  20:35:04
19 processes:  1 running, 18 sleeping
CPU:  0.0% user,  0.0% nice, 25.0% system,  0.0% interrupt, 75.0% idle
Mem: 13M Active, 14M Inact, 137M Wired, 3749M Free
ARC: 28M Total, 6646K MFU, 20M MRU, 233K Header, 1196K Other
     13M Compressed, 38M Uncompressed, 2.95:1 Ratio
Swap: 2048M Total, 2048M Free

  PID USERNAME    THR PRI NICE   SIZE    RES STATE    C   TIME    WCPU COMMAND
  857 root          1  40    0    14M  3528K CPU2     2   0:00   0.02% top
  841 root          1   8    0    13M  3180K wait     2   0:00   0.00% login
  716 root          1  40    0    13M  2788K select   2   0:00   0.00% syslogd
  853 fbonino       1   8    0    13M  3152K wait     2   0:00   0.00% su
  854 root          1  16    0    14M  3860K pause    2   0:00   0.00% csh
  850 fbonino       1   8    0    13M  3300K wait     2   0:00   0.00% sh
  848 root          1  47    0    13M  2292K ttyin    2   0:00   0.00% getty
  845 root          1  47    0    13M  2288K ttyin    3   0:00   0.00% getty
  843 root          1  47    0    13M  2288K ttyin    1   0:00   0.00% getty
  844 root          1  47    0    13M  2296K ttyin    1   0:00   0.00% getty
  846 root          1  47    0    13M  2292K ttyin    1   0:00   0.00% getty
  847 root          1  47    0    13M  2288K ttyin    1   0:00   0.00% getty
  849 root          1  47    0    13M  2288K ttyin    1   0:00   0.00% getty
  829 root          1   8    0    13M  2532K nanslp   2   0:00   0.00% cron
  433 root          1   4    0    13M  2740K select   3   0:00   0.00% dhclient
  430 root          1  40    0    13M  2624K select   1   0:00   0.00% dhclient
```

Figura 8: Análisis de carga del sistema por defecto

Partiendo de estos datos, se decidió escalar en la carga del sistema con procesos `USER` mediante el comando de pruebas `stress` para simular procesos de carga intensiva, y un script custom que lance procesos sencillos que simulen carga en la CPU:

```
1  #!/bin/sh
2  num_procs=10
3  cpu_load_cmd="for _ in $(seq 1 1000000000); do :; done"
4  for i in $(seq 1 $num_procs); do
5        sh -c $cpu_load_cmd &
6  done
7  wait
```



Se comenzó corriendo el script una vez para simular 10 procesos `USER` ligeros, y obtuvimos una carga de poco más del 30%:

```
last pid:  1306;  load averages:  1.67,  1.29,  1.11    up 0+01:01:10  21:32:12
23 processes:  2 running, 21 sleeping
CPU: 34.3% user,  0.0% nice, 28.4% system,  0.0% interrupt, 37.3% idle
Mem: 788M Active, 1040M Inact, 159M Wired, 1927M Free
ARC: 30M Total, 6990K MFU, 22M MRU, 266K Header, 1441K Other
     15M Compressed, 43M Uncompressed, 2.96:1 Ratio
Swap: 2048M Total, 2048M Free

  PID USERNAME    THR PRI NICE   SIZE    RES STATE    C   TIME    WCPU COMMAND
 1305 root          1  53    0    13M  2284K CPU1     1   0:13 221.93% seq
 1304 root          1  -8    0  2587M  1801M piperd   2   0:06 106.78% sh
 1306 root          1  40    0    14M  3588K CPU3     3   0:00   0.03% top
 1293 fbonino       1  40    0    14M  3612K select   1   0:00   0.02% top
 1289 root          1  40    0    14M  3640K select   3   0:00   0.01% top
  716 root          1  40    0    13M  2804K select   2   0:00   0.00% syslogd
  897 root          1  16    0    14M  3968K pause    2   0:00   0.00% csh
 1033 root          1  16    0    14M  3956K pause    1   0:00   0.00% csh
  829 root          1   8    0    13M  2532K nanslp   1   0:00   0.00% cron
  844 root          1   8    0    13M  3216K wait     2   0:00   0.00% login
  841 root          1   8    0    13M  3180K wait     2   0:00   0.00% login
  850 fbonino       1   8    0    13M  3304K wait     2   0:00   0.00% sh
  843 root          1   8    0    13M  3200K wait     3   0:00   0.00% login
  511 root          1  40    0    11M  1552K select   3   0:00   0.00% devd
 1022 fbonino       1   8    0    13M  3368K wait     2   0:00   0.00% sh
 1031 fbonino       1   8    0    13M  3152K wait     2   0:00   0.00% su
```

Figura 9: Análisis de carga del sistema, prueba 1

Correr el script 3 veces (30 procesos simultáneos de carga ligera) suponía una carga mayor, pero al ser procesos ligeros vemos que apenas casi duplica el consumo para 10 procesos:

```
last pid:   858;  load averages:  1.57,  0.94,  0.43    up 0+00:05:23  21:43:00
23 processes:  4 running, 19 sleeping
CPU: 67.5% user,  0.0% nice, 32.5% system,  0.0% interrupt,  0.0% idle
Mem: 504M Active, 632M Inact, 151M Wired, 2627M Free
ARC: 28M Total, 5768K MFU, 21M MRU, 132K Anon, 225K Header, 1174K Other
     13M Compressed, 39M Uncompressed, 2.96:1 Ratio
Swap: 2048M Total, 2048M Free

  PID USERNAME    THR PRI NICE   SIZE    RES STATE    C   TIME    WCPU COMMAND
  857 fbonino       1  45    0    13M  2292K RUN      0   0:02 501.01% seq
  855 fbonino       1  46    0    13M  2296K CPU0     0   0:03 478.02% seq
  853 fbonino       1  46    0    13M  2292K CPU1     1   0:03 361.88% seq
  854 fbonino       1  -8    0   659M   420M piperd   1   0:01 235.79% sh
  856 fbonino       1  -8    0   333M   251M piperd   0   0:01 224.63% sh
  852 fbonino       1  -8    0   659M   448M piperd   0   0:02 183.58% sh
  858 fbonino       1  40    0    14M  3592K CPU2     2   0:00   0.58% top
  697 root          1  40    0    13M  2792K select   2   0:00   0.01% syslogd
  810 root          1   8    0    13M  2540K nanslp   2   0:00   0.00% cron
  831 fbonino       1   8    0    13M  3304K wait     2   0:00   0.00% sh
  823 root          1   8    0    13M  3180K wait     2   0:00   0.00% login
  825 root          1  43    0    13M  2288K ttyin    1   0:00   0.00% getty
  830 root          1  43    0    13M  2292K ttyin    1   0:00   0.00% getty
  828 root          1  43    0    13M  2292K ttyin    2   0:00   0.00% getty
  824 root          1  43    0    13M  2280K ttyin    1   0:00   0.00% getty
  827 root          1  43    0    13M  2292K ttyin    0   0:00   0.00% getty
```

Figura 10: Análisis de carga del sistema, prueba 2



Por otro lado, correr dos procesos de alto consumo mediante el comando `stress -c 2` nos dio un porcentaje del 50%:

```
last pid:  1054;  load averages:  3.22,  3.24,  2.99    up 0+00:31:17  22:08:54
22 processes:  3 running, 19 sleeping
CPU: 50.0% user,  0.0% nice, 25.0% system,  0.0% interrupt, 25.0% idle
Mem: 9640K Active, 60M Inact, 36K Laundry, 184M Wired, 3660M Free
ARC: 74M Total, 14M MFU, 58M MRU, 377K Header, 1361K Other
     58M Compressed, 111M Uncompressed, 1.93:1 Ratio
Swap: 2048M Total, 10M Used, 2038M Free

  PID USERNAME    THR PRI NICE   SIZE    RES STATE    C   TIME    WCPU COMMAND
 1053 root          1  60    0    13M  1984K CPU2     2   0:08 302.05% stress
 1054 root          1  60    0    13M  1984K CPU1     1   0:08 302.05% stress
 1052 root          1  41    0    14M  3120K CPU0     0   0:00   0.06% top
  878 root          1  16    0    14M  2376K pause    2   0:00   0.00% csh
  697 root          1  40    0    13M  1764K select   2   0:00   0.00% syslogd
  831 fbonino       1   8    0    13M  1252K wait     2   0:00   0.00% sh
  492 root          1  40    0    11M  1120K select   2   0:00   0.00% devd
  823 root          1   8    0    13M   980K wait     2   0:00   0.00% login
  491 _dhcp         1  40    0    13M  1648K select   0   0:00   0.00% dhclient
  810 root          1   8    0    13M  1244K nanslp   0   0:00   0.00% cron
  877 fbonino       1   8    0    13M   980K wait     2   0:00   0.00% su
  431 root          1   4    0    13M  1388K select   0   0:00   0.00% dhclient
  428 root          1  40    0    13M  1468K select   2   0:00   0.00% dhclient
 1051 root          1   8    0    13M  1984K wait     0   0:00   0.00% stress
  825 root          1  43    0    13M   972K ttyin    1   0:00   0.00% getty
  830 root          1  43    0    13M   976K ttyin    1   0:00   0.00% getty
```

Figura 11: Análisis de carga del sistema, prueba 3

Con estas estadísticas tenidas en cuenta, tomamos la siguiente escala de referencia para identificar la intensidad del consumo de recursos de la CPU en el sistema:

Porcentaje de uso de la CPU	Intensidad de carga
x < 30%	LOAD_IDLE
30% ≤ x < 45%	LOAD_LOW
45% ≤ x < 75%	LOAD_NORMAL
75% ≤ x < 85%	LOAD_HIGH
85% ≤ x < 98%	LOAD_INTENSE
98% ≤ x	LOAD_SEVERE

Paralelamente se desarrolló un segundo módulo encargado de recopilar estadísticas, pero esta vez a nivel de hilos de ejecución. Este módulo `thread_stats` es similar al ya desarrollado: recopila estadísticas a través de las macros `FOREACH_PROC_IN_SYSTEM` y `FOREACH_THREAD_IN_PROC`. Con la variable `ts_estcpu`[7] de la estructura de los hilos podemos saber cuáles tuvieron uso de CPU últimamente

7 *Variable que se obtiene de la estructura `td_sched` definida por el scheduler que indica el consumo estimado de CPU por el hilo. Este valor tiene un decaimiento definido a medida que pasa el tiempo.*



y, por lo tanto, detectar la cantidad de hilos y procesos activos además de cuánto CPU se destinó a procesos activos y cuánto a procesos `IDLE`[8].

Además, para facilitar la comunicación de información proveniente de los procesos y sus hilos, se tomó la decisión de darle a este módulo la tarea de obtener y procesar datos sobre los tipos de procesos del sistema según la necesidad de procesamiento que tengan: baja performance, estándar, alta performance y críticos, y si solicitan la monopolización de un núcleo de la CPU o no. Esta información la proporcionan los mismos procesos mediante un *payload* específico de información para el *scheduler* incluido en la *metadata* de su ejecutable ELF. El módulo lee la *metadata*, procesa la cantidad, los tipos de procesos en ejecución y si requieren monopolización de núcleo. Si algún proceso no proporciona dicha etiqueta, por defecto es tratado como un proceso estándar y que no requiere monopolizar un núcleo.

Finalmente, el módulo carga toda la información sobre los procesos y sus hilos de ejecución en la estructura `threads_stats` que será compartida para que otro módulo se encargue de su procesamiento.

4.3.3 Análisis de los resultados

Con las estadísticas recopiladas, almacenadas y leídas exitosamente, y habiendo indagado lo necesario en el sistema operativo y las *syscalls* disponibles para obtener las estadísticas de uso a nivel de núcleos de la CPU y a nivel de hilos y procesos, dimos por cerrada esta iteración exitosamente.

4.3.4 Próximos pasos

El siguiente paso en el desarrollo de este proyecto es el de preparar el sistema para darle una utilidad a estas estadísticas recopiladas. Es por esto que se continuará con la integración de las funcionalidades de encendido/apagado de núcleos y monopolización de los mismos al sistema para más adelante aprovechar la información obtenida y estas funcionalidades.

8 *Proceso que no está realizando ninguna tarea activa. Generalmente se crean para ocupar la CPU cuando no hay otras tareas más prioritarias que realizar, permitiendo que el sistema operativo utilice la capacidad de procesamiento de la CPU de manera eficiente cuando no hay otras tareas de mayor prioridad.*

4.4 Cuarta iteración

El proyecto *Optimización del planificador a corto plazo con redes de Petri* aporta dos funcionalidades al *kernel*: la capacidad de encender/apagar los núcleos de la CPU del sistema, y la posibilidad de que un hilo pueda monopolizar un núcleo de la CPU, sirviéndose del modelo de *scheduler* con red de Petri trabajado hasta el momento.

4.4.1 Objetivos

El objetivo de esta iteración es el de la integración de los módulos mencionados a nuestro proyecto integrador en la versión 13.2 de FreeBSD.

4.4.2 Implementación

Para comenzar a incorporar el módulo de encendido/apagado de núcleos, se agregaron las plazas y transiciones correspondientes a la red:

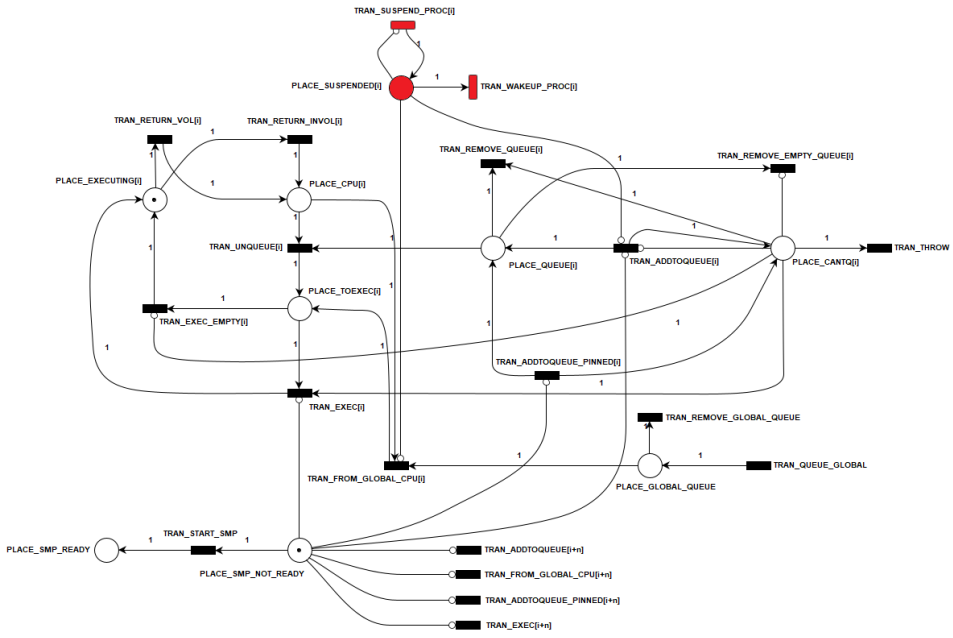


Figura 12: Red de Petri con suspensión de núcleo

El siguiente paso fue modificar los archivos `sched_4bsd.c`, `petri_global_net.c` y `sched_petri.h` con el código desarrollado por los autores del módulo y recompilar el *kernel* con estos cambios. Una vez obtenido el nuevo *kernel*, se



hizo un pequeño módulo que llama a la función `toggle_active_cpu`, la cual conmuta el estado de actividad del núcleo de la CPU que se le indique (exceptuando el núcleo 0) y se pudo ver cómo efectivamente los núcleos dejan de tener carga de procesos al deshabilitarlos hasta que se los reincorpora.

Observando el funcionamiento de este módulo, notamos que había que revisar el funcionamiento de la transición `THROW` en la red. Esta transición se encarga de equilibrar el encolado de hilos en los núcleos implementando un algoritmo de tipo *round robin*[9]. En el anexo A.1 se muestra, sobre una red simplificada, cómo sería el funcionamiento de esta transición en un sistema de cuatro núcleos: cuando un hilo es ubicado en la cola de un núcleo, éste no puede encolar más hasta que los restantes hagan lo mismo, y la transición `THROW` es disparada llevándose los *tokens* que inhiben el encolado.

Al incorporar este módulo nuevo, se detectó que un núcleo en estado `SUSPENDED` no agrega *tokens* en la plaza que permite disparar la transición `THROW` cuando esté sensibilizada (todos los núcleos tienen un *token* en esa plaza) y por lo tanto se llega a un punto en el que ningún núcleo puede encolar hilos. En el anexo A.2 se puede observar la secuencia que describe este suceso.

9 *Algoritmo simple de distribución de recursos de CPU que asigna un intervalo de tiempo fijo a cada proceso en un sistema operativo, luego del cual el proceso en ejecución queda suspendido y el siguiente proceso en la cola pasa a ejecutarse.*

Para solucionar esto, se sopesaron algunas alternativas, siendo la elegida la de implementar una estructura **OR** en la red, para que la transición `THROW` sea disparada cuando ya todos los núcleos hayan encolado hilos o se encuentren en estado `SUSPENDED`:

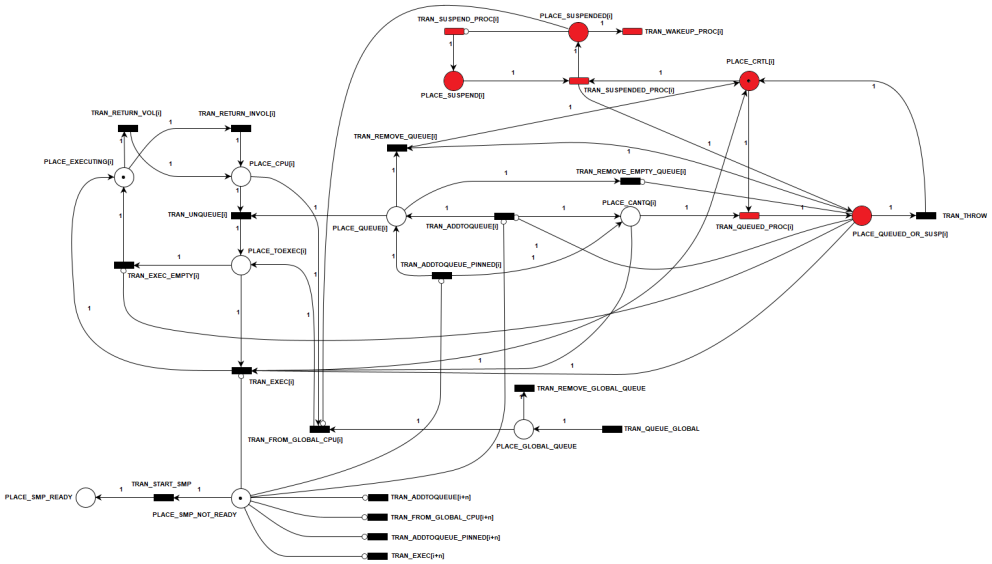


Figura 13: Red de Petri corregida

Ahora, como se ve mediante una red simplificada en el anexo A.3, cuando se inicia el proceso de suspendido de un núcleo disparando `TRAN_SUSPEND_PROC`, se dispara también la transición automática `TRAN_SUSPENDED_PROC` que llena la plaza que marca el estado suspendido del core (`PLACE_SUSPENDED`) y la que inhibe el encolado (`PLACE_QUEUED_OR_SUSP`). Esta última plaza se llena en este caso y también cuando el núcleo encola un hilo, por lo que permite a la transición `THROW` volver a cumplir su función de equilibrado.

Para adaptar estos cambios al código sólo hubo que actualizar las matrices de incidencia e inhibición de la red de recursos, agregar las nuevas transiciones y plazas, y agregar dos nuevas transiciones automáticas: las que manejan si un núcleo está encolando o está suspendido (relacionadas a la transición `THROW`, que era la única automática hasta aquí). También se revisó la función `toggle_active_cpu` y se agregó que debe validar si el modo SMP está activado para poder suspender núcleos, sino se disparan *kernel panics*. También se crearon 2 funciones para tener más control sobre el estado de los núcleos, llamadas `turn_on_cpu` y `turn_off_cpu` que realizan su cometido llamando a `toggle_active_cpu`.



Con respecto al módulo de monopolización de núcleos de la CPU, se aplicaron las modificaciones realizadas por sus autores en el código de los archivos `sched_4bsd.c`, `sched_petri.c`, `sched_petri.h` y `petri_global_net.c` y se recompiló el *kernel* con estos cambios. Para probar el correcto funcionamiento de esta integración, se instaló un *driver* de *kernel* que llama a la función encargada de asociar un CPU a un hilo.

Por último, se realizaron unas modificaciones sobre esta funcionalidad, cambiando la forma de monopolizar: anteriormente un hilo adquiría un núcleo de la CPU, ahora lo hace un proceso (y todos sus hilos) ya que la forma de obtener información sobre si se requiere utilizar este módulo es a través de información en la *metadata* de los procesos. Además se agregaron funciones (`monopolize_cpu` y `release_cpu`) que son *wrappers* de `toggle_pin_cpu_to_proc` para mejorar el control del módulo.

4.4.3 Análisis de los resultados

Se pudieron integrar correctamente estos módulos al sistema, obteniendo un *kernel* funcional y estable que, sirviéndose de la red de Petri, ahora nos permite: habilitar o deshabilitar el encolado de hilos en los distintos núcleos de la CPU y así controlar la carga de los mismos, y que un hilo tome el control de un núcleo de la CPU, monopolizándolo. Aquí se puede observar cómo el núcleo 1 de la CPU se deshabilitó, quedando sin recibir más hilos para encolar:

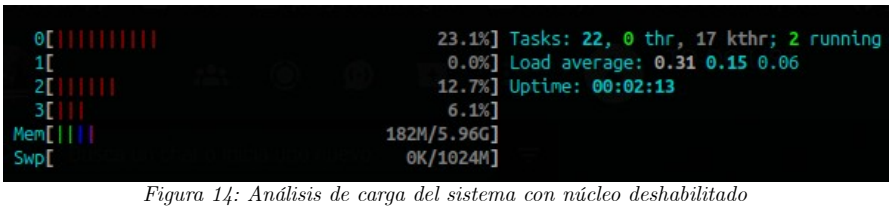


Figura 14: Análisis de carga del sistema con núcleo deshabilitado

En las siguientes capturas de pantalla se ve representado el comportamiento del sistema cuando se ejecuta un programa de cálculo de números primos con *metadata* insertada para monopolizar un CPU:



Figura 15: Cálculo de números primos con monopolización

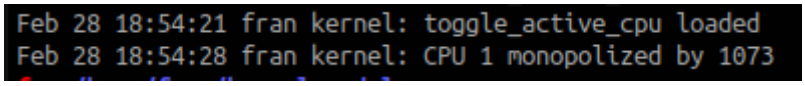


Figura 16: Log de monopolización exitosa para cálculo de números primos

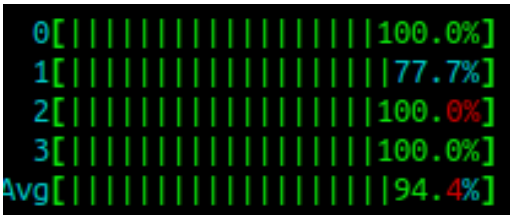


Figura 17: Monitorización de recursos consumidos con monopolización de alta performance

Aquí vemos otro ejemplo similar, con la diferencia de que el programa ejecutado no realiza uso de CPU (realiza un *sleep*):



Figura 18: Ejecución de sleep

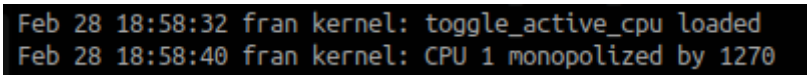


Figura 19: Log de monopolización exitosa para ejecución de sleep

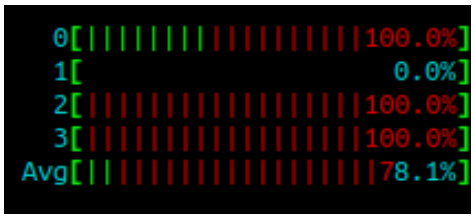


Figura 20: Monitorización de recursos consumidos con monopolización de baja performance

Se observa cómo la red queda libre de interbloqueos, no acotada debido a las plazas de encolado, e insegura:

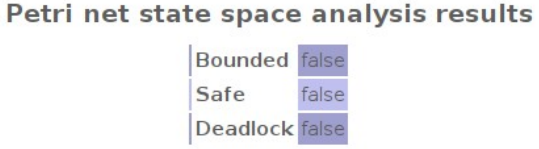


Figura 21: Análisis de la red de Petri modificada



4.4.4 Próximos pasos

Con estas modificaciones podemos aprovechar las capacidades que nos proveen estos módulos para poder gestionar el estado de los núcleos bajo el control del sistema operativo. Por esto, el siguiente paso del proyecto es desarrollar una utilidad que procese estadísticas de uso e información provista por los procesos a través de su *metadata*, y aplicando un criterio pueda tomar decisiones para así aprovechar las funcionalidades de gestión de núcleos.



4.5 Quinta iteración

4.5.1 Objetivos

Se tomó como objetivo para esta iteración el procesamiento de las estadísticas obtenidas y la toma de decisiones en base a las mismas teniendo correctamente adaptado el módulo que controla el cambio de estado de actividad de los núcleos de la CPU.

4.5.2 Implementación

El primer paso para el desarrollo de este módulo fue leer mediante `kernel_sysctlbyname` la información publicada como *string* usando `sysctl` por parte de los módulos colectores de estadísticas, y escribirla en un archivo de *log* del módulo. Como el funcionamiento de este módulo depende de los módulos `cpu_stats` y `thread_stats`, se declaró mediante la *macro* `MODULE_DEPEND` que este módulo es dependiente de los mencionados (lo que ocasiona que el mismo no pueda ser cargado en el sistema si primero no se cargaron los dos indicados, y que estos últimos no puedan ser desinstalados antes que este).

Una vez obtenidas las métricas, se procedió a *parsear* los *strings* y obtener los datos de interés para poder empezar a hacer cálculos que nos ayuden en la toma de decisiones. Para poder cumplir el cometido de medir la carga del sistema y las necesidades de los procesos para manejar el estado de actividad de los núcleos, fue necesario obtener lo siguiente:

- *Carga de usuario del sistema*: Obtener los *ticks* de cada núcleo destinados a procesos de usuario en el último tiempo para determinar la carga del sistema.
- *Necesidades de cada proceso*: Evaluar las necesidades que el sistema debería "reservar" para los procesos según la información que estos proveen a través de etiquetas pasadas vía *metadata* en sus ejecutables.
- *Núcleo más ocioso*: Con las métricas obtenidas, determinar cuál es el núcleo con menor actividad para apagarlo en caso de que el estado del sistema lo requiera.



El módulo consiste de 2 funciones *callback* principales que son pasadas a la API *callout* para poder ser llamadas periódicamente. Estas son *callback_ load*, llamada cada 30 segundos, y *callback_ monopolization*, llamada cada 1 segundo:

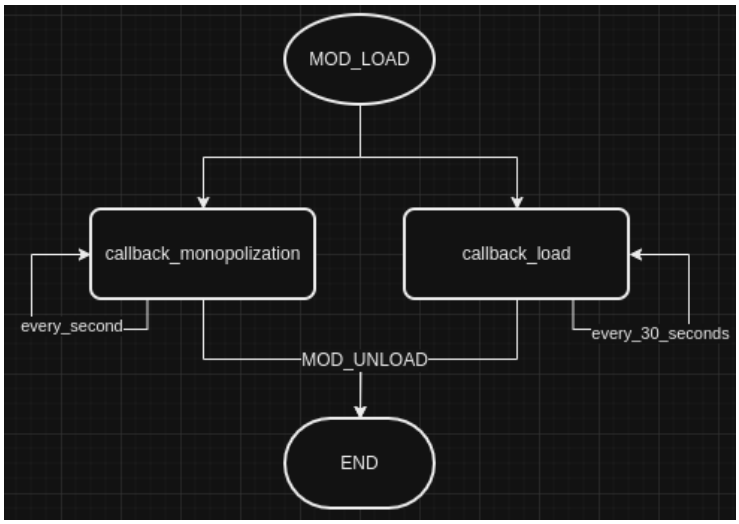


Figura 22: Flujo de llamadas a callbacks

4.5.2.1 `callback_load`

1. Obtenemos las estadísticas recolectadas por los módulos `cpu_stats` y `thread_stats`.

2. Calculamos la carga de usuario y el nivel de las necesidades de los procesos en el sistema.

   a) Para la carga de usuario se calcula el porcentaje de uso de cada núcleo y se promedia para obtener, según este porcentaje, uno de los valores definidos en la Tercera iteración: `LOAD_IDLE`, `LOAD_LOW`, `LOAD_NORMAL`, `LOAD_HIGH`, `LOAD_INTENSE` y `LOAD_SEVERE`.

   b) Con la cantidad de cada uno de los distintos tipos de procesos (`LOWPERF`, `STANDARD`, `HIGHPERF` y `CRITICAL`) se diseñó un sistema de *scoring* a partir de los valores mencionados en la tercera iteración. Este *score* cae siempre entre 0 y 100, por lo que el valor de las necesidades de los procesos será uno de los 6 niveles de carga definidos.

3. Dado que la carga de usuario del sistema representa la realidad de los núcleos en el momento, y las necesidades de los procesos son requerimientos, se decidió darle más peso a lo primero para la toma de decisiones sobre el apagado/encendido de los núcleos.



4. Se decidió que cuando el nivel de carga (calculado a partir del uso y las necesidades) sea menor a `LOAD_NORMAL` (es decir, cuando el nivel de carga sea `LOAD_IDLE` o `LOAD_LOW`), se busque el núcleo más ocioso, calculando el porcentaje de *ticks* en modo `IDLE` de los mismos, y se apague.

    - Para la toma de esta decisión existe un *delay* de acción para chequear que sea consistente el *no uso* del sistema, y evitar apagar núcleos por error.
    - Para evitar conflictos, se decidió que si existen procesos queriendo monopolizar núcleos, no se pueda suspender.
    - Se limita la cantidad de núcleos que se pueden apagar ya que de lo contrario, al haber procesos del sistema que siguen corriendo, el mismo puede quedar inutilizable.

5. Si el nivel de carga obtenido es mayor o igual a `LOAD_NORMAL`, en caso de haber algún núcleo de la CPU apagado se lo enciende y también se acorta el tiempo de chequeo de nivel de carga para que, en caso de necesitar encender más núcleos la reacción sea más rápida.



Figura 23: Secuencia de `callback_load`

4.5.2.2 `callback_monopolization`

1. Se obtiene la información sobre monopolización de los procesos consultando al módulo que la recopila.
2. Se consulta el estado de monopolización de los cores en la red de Petri.
3. De haber procesos monopolizando núcleos, se revisa si deben liberarlos por no requerirlos más o por si existe algún proceso prioritario que solicite monopolizar.
4. De haber procesos requiriendo núcleos, se le asigna a los de mayor prioridad núcleos disponibles para monopolizar.
    - Al igual que con la cantidad de cores que pueden suspenderse, el número de cores monopolizables está limitado.

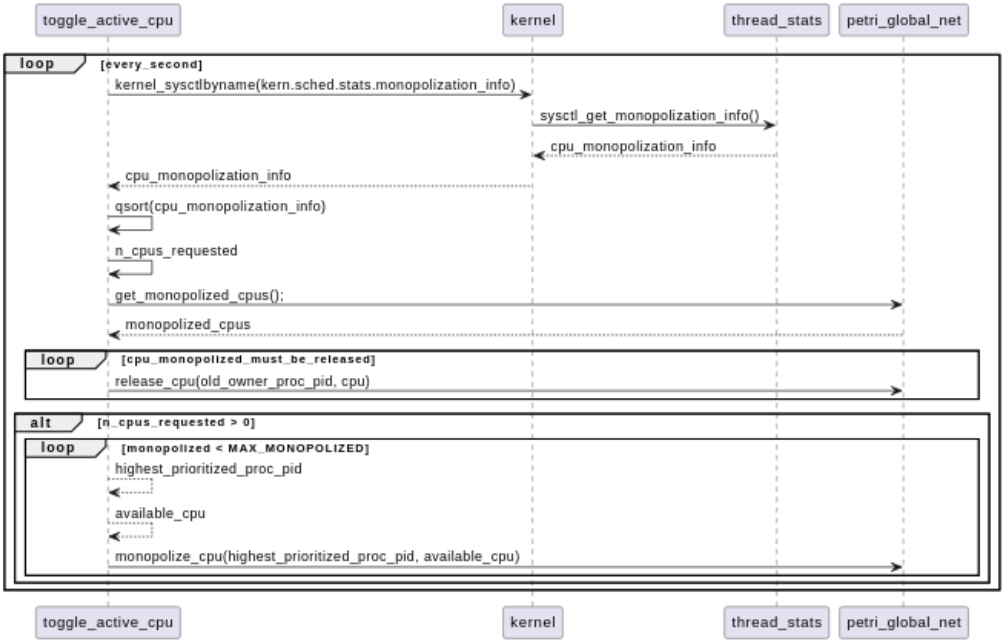


Figura 24: Secuencia de `callback_monopolization`

4.5.3 Análisis de los resultados

Se logró un primer acercamiento a la gestión de núcleos en base a estadísticas en tiempo real del sistema e información proporcionada por los procesos sobre sus necesidades. Este acercamiento es centrado en funcionalidad, dejando de lado la búsqueda de gran *performance*.

En las siguientes imágenes observamos el funcionamiento del módulo gestor:

- Primero se cargan los módulos necesarios en el *kernel* y se deja el *servidor* en estado *idle*. Esto es detectado por el módulo gestor a través de las estadísticas que le sirven y se encarga de suspender un núcleo. Se observa cómo el módulo gestor *loguea* que suspende el CPU 1 y el mismo no presenta actividad:

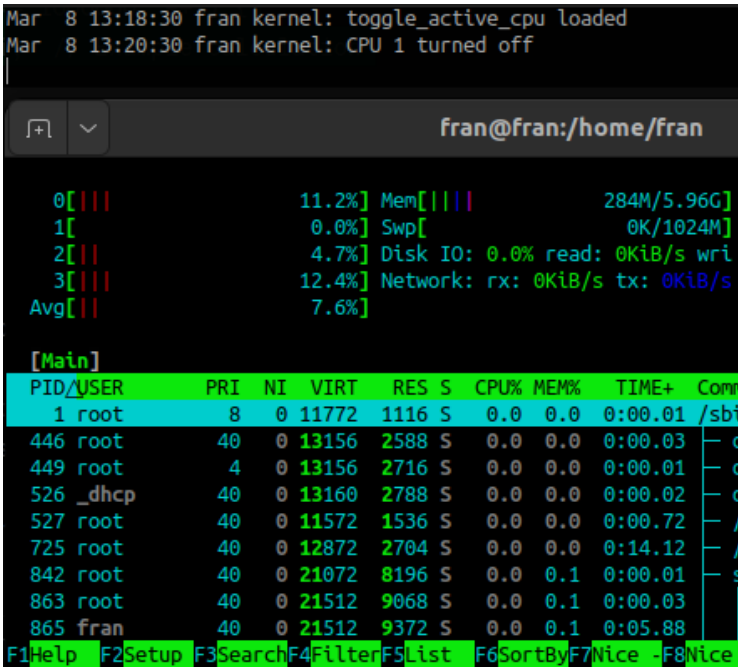


Figura 25: Monitorización de comportamiento del módulo gestor

- Luego, se procede a ejecutar un proceso que solicita monopolizar un núcleo. En este punto, el sistema reacciona primero encendiendo el núcleo suspendido y luego asignando un núcleo al proceso que lo requirió. A continuación, se ejecuta un proceso que también quiere monopolizar un núcleo, y al ser de mayor prioridad y estando el máximo posible de cores monopolizados (para este caso con 4 núcleos en total), el sistema gestor le quita el núcleo al proceso anterior y se

lo asigna al nuevo. Estas acciones realizadas por el módulo son escritas en un archivo de *log*:

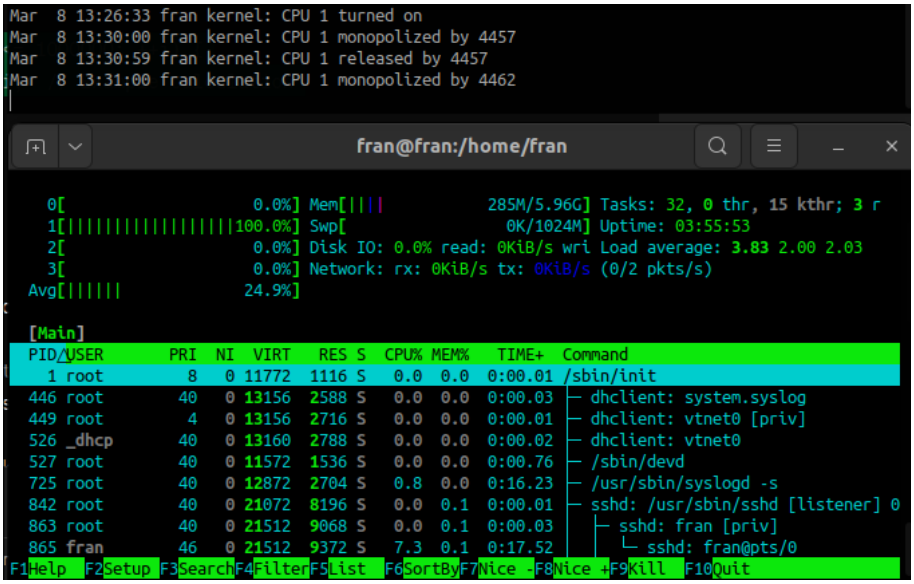


Figura 26: Monitorización del comportamiento del módulo gestor con múltiples monopolizaciones

4.5.4 Próximos pasos

Con el aumento de la cantidad de líneas de código escritas para las diferentes integraciones e implementaciones desarrolladas, se decidió revisar el trabajo y tratar de refactorizar el código. Esto se debe a que al ser un proyecto a nivel de *kernel*, es preciso aumentar la legibilidad del código y optimizar procesos.



4.6 Sexta iteración

Las buenas prácticas de programación son esenciales para garantizar la eficiencia, mantenibilidad, escalabilidad y estabilidad del código. Esto aplica a programas simples de usuario, módulos de *kernel*, código de sistemas operativos, páginas web y básicamente todo lo que involucre programación. El código limpio no sólo mejora la legibilidad, facilitando la comprensión del funcionamiento de lo que se desarrolla, sino que también reduce la probabilidad de errores y facilita la depuración de los problemas que puedan surgir. La refactorización periódica es crucial, ya que permite adaptarse a cambios en los requisitos y tecnologías emergentes, manteniendo un código óptimo, claro y libre de posibles vulnerabilidades, por ejemplo, de seguridad. Este enfoque no sólo promueve la calidad de lo desarrollado, sino que también contribuye a la robustez y seguridad en su conjunto.

4.6.1 Objetivos

Se propuso como objetivo de esta iteración la refactorización del proyecto integrador para mejorar la calidad y organizar mejor los módulos de *kernel* desarrollados y los de terceros que fueron integrados y adaptados.

4.6.2 Implementación

4.6.2.1 Escalabilidad

En esta etapa, el proyecto funcionaba correctamente con el modelo propuesto para una CPU de cuatro núcleos. De acuerdo a esto, lo primero que se detectó fue la necesidad de escalarlo a sistemas con un número mayor de núcleos, ya que las computadoras promedio a la fecha superan fácilmente esa cifra, llegando a tener 8 y hasta 16 núcleos. Computadoras dedicadas a tareas de alto procesamiento en sistemas especializados, como los servidores, pueden llegar a superar los 100 núcleos de procesamiento. Por otro lado, forzar un *scheduler* a trabajar con 4 núcleos podría traer problemas y tener un comportamiento indeterminado en sistemas mononúcleo o con una cantidad de núcleos distinta a 4. Para este propósito se exploraron varias implementaciones posibles.

La primera alternativa considerada consistía en hacer uso de la función `kvm_getncpus` definida en `lib/libkvm/kvm.h` (cuyo cuerpo está en `lib/libkvm/kvm_pcpu.c`) que devuelve el valor de `mp_ncpus`, una variable resuelta en tiempo de ejecución que contiene el número de núcleos de la CPU. Esta función forma



parte de una interfaz de comunicación entre las aplicaciones de nivel de usuario y el *kernel* del sistema, otorgando la posibilidad de acceder a memoria del *kernel* para examinar (y en ciertos casos manipular) estructuras de datos a este nivel. Sin embargo, rápidamente fue descartada debido a que el llamado a esta función involucra un *path* a la imagen del *kernel*, e implica entablar una comunicación con el *kernel* desde el espacio de usuario en lugar de operar en el espacio de *kernel* en sí. Esto nos dio indicios de que no sería la mejor alternativa, ya que hacer que la obtención de este campo dependa de la ubicación y nombre específico del archivo de *kernel* donde se está ejecutando podría traer problemas en el tiempo de procesamiento y en la escalabilidad del proyecto. Además, operar desde el espacio de usuario hacia el espacio de *kernel* implica que las operaciones no se ejecutarán con suficiente rapidez. Buscábamos una opción simple y rápida para obtener este valor.

Una segunda alternativa fue la de hacer uso de la estructura `cpu_group` definida en `/usr/src/sys/sys/smp.h`, la cual representa el conjunto de núcleos de las CPUs del sistema con sus características. Esta estructura posee un campo `cg_count` que indica el número total de núcleos que posee el sistema. Se buscó implementar en la librería común a los módulos una macro que devuelva este campo, de la siguiente forma:

```
1  #include <sys/smp.h>
2  #define CPU_NUMBER cpu_group_t->cg_count
```

Sin embargo, remitiéndonos a la definición de *macro*, se llegó a la conclusión de que esto no funcionaría (y, de hecho, no funcionó) ya que una *macro* se debe resolver estrictamente en tiempo de compilación, y el valor de `cg_count` está directamente ligado a `mp_ncpus`, que se resuelve en tiempo de ejecución como vimos anteriormente. Esta alternativa también fue descartada debido a que era complicado e impráctico incluir el archivo `/usr/src/sys/sys/smp.h` en los lugares donde se haga uso del número de núcleos de la CPU, y porque no sería escalable para sistemas multiprocesador.

La mejor alternativa hallada fue la de definir en el archivo *header* del *scheduler* basado en la red de Petri una variable `CPU_NUMBER` que almacene el valor de `mp_ncpus` para usar de manera directa, y una función `init_resource_net` que se encargue de inicializar los recursos necesarios para la definición de la red; entre ellos, el número de núcleos de la CPU. Esta función es llamada en la etapa de *setup* del *scheduler* 4BSD, donde se define el valor de `mp_ncpus` de manera interna al sistema operativo y se define todo lo pertinente al *scheduler*. Dentro de nuestra función definimos todas las variables de la red de Petri que dependen del número de núcleos



de la CPU. A continuación se muestran los fragmentos de código implementados para esta solución:

```
1 static void sched_setup(void *dummy) {
2   setup_runqs();
3   init_resource_net(); // inicialización de los recursos de la red
4   sched_load_add();
5 }
```

```
1  void init_global_variables(void) {
2    CPU_NUMBER = mp_ncpus;
3    CPU_NUMBER_PLACES = (CPU_BASE_PLACES*CPU_NUMBER) + GLOBAL_PLACES;
4    CPU_NUMBER_TRANSITIONS = (CPU_BASE_TRANSITIONS*CPU_NUMBER) + GLOBAL_TRANSITIONS;
5    PER_CPU_LAST_TRANSITION = (CPU_NUMBER_TRANSITIONS - GLOBAL_TRANSITIONS);
6    PER_CPU_TOTAL_AUTO_TRANSITIONS = (CPU_AUTO_TRANSITIONS*CPU_NUMBER);
7    AUTOMATIC_TRANSITIONS = GLOBAL_AUTO_TRANSITIONS + PER_CPU_TOTAL_AUTO_TRANSITIONS;
8    PLACE_GLOBAL_QUEUE = (CPU_NUMBER_PLACES - 3);
9    PLACE_SMP_NOT_READY = (CPU_NUMBER_PLACES - 2);
10   PLACE_SMP_READY = (CPU_NUMBER_PLACES - 1);
11   TRAN_REMOVE_GLOBAL_QUEUE = (CPU_NUMBER_TRANSITIONS - 4);
12   TRAN_START_SMP = (CPU_NUMBER_TRANSITIONS - 3);
13   TRAN_THROW = (CPU_NUMBER_TRANSITIONS - 2);
14   TRAN_QUEUE_GLOBAL = (CPU_NUMBER_TRANSITIONS - 1);
15 }
```

4.6.2.2 Macros

Otro punto importante fue la incorporación de *macros* para mejorar la reutilización de porciones comunes de código. Algunas de las *macros* útiles implementadas para la obtención de elementos de la red de Petri definidas en `sched_petri.h` se muestran a continuación:

```
1  #define CPU_BASE_TRANSITION(cpu) cpu * CPU_BASE_TRANSITIONS
2  #define CPU_BASE_PLACE(cpu) cpu * CPU_BASE_PLACES
3  #define CPU_NUMBER_TRANSITIONS (CPU_BASE_TRANSITIONS*CPU_NUMBER) + 4
4  #define PER_CPU_LAST_TRANSITION CPU_NUMBER_TRANSITIONS - 4
5  #define TRAN_REMOVE_GLOBAL_QUEUE (CPU_NUMBER_TRANSITIONS - 4)
6  #define TRAN_START_SMP (CPU_NUMBER_TRANSITIONS - 3)
7  #define TRAN_THROW (CPU_NUMBER_TRANSITIONS - 2)
8  #define TRAN_QUEUE_GLOBAL (CPU_NUMBER_TRANSITIONS - 1)
9  #define TRANSITION(cpu, transition) (CPU_BASE_TRANSITION(cpu) + transition)
10 #define PLACE(cpu, place) (CPU_BASE_PLACE(cpu) + place)
```



De esta manera logramos reducir múltiples líneas de código a lo largo del proyecto, incrementado a la par la legibilidad del mismo. Un claro ejemplo es el siguiente caso en el que pudimos pasar de expresiones como esta:

```
1 place = num_place + (cpu_n * CPU_BASE_PLACES);
2 transition = num_transition + (cpu_n * CPU_BASE_TRANSITIONS);
```

a otras más fáciles de interpretar:

```
1 place = PLACE(cpu_n, num_place);
2 transition = TRANSITION(cpu_n, num_transition);
```

4.6.2.3 Estructuras

Dada la necesidad de mantener todo lo desarrollado a nivel de *kernel* para reducir al máximo el tiempo de procesamiento, se optó por definir las estructuras para almacenar las estadísticas de uso de los núcleos de la CPU, pudiendo así *setear* sus valores de manera más intuitiva en el módulo encargado de obtenerlas, y pudiendo leerlas más rápidamente en el módulo encargado de tomar decisiones en base a ellas. Se hizo algo similar para las estadísticas a nivel de hilos de ejecución:

```
1  struct cpu_core_stats {
2    long ticks[CPUSTATES];
3    long delta[CPUSTATES];
4    long ticks_total;
5    long delta_total;
6  };
7
8  struct threads_stats {
9    int estcpu_idle;
10   int estcpu_work;
11   int threads_in_system;
12   int threads_active;
13   int procs_in_system;
14   int procs_active;
15   int proc_needs[N_PROC_NEEDS];
16 };
```

4.6.2.4 Estadísticas

Cuando se realizaron los módulos de recopilación de estadísticas, estos calculaban las métricas y las imprimían en un archivo de *log*. Ese mismo arreglo de caracteres que se imprimía era guardado en un *string* que se publicaba al sistema mediante la utilidad `sysctl`, más precisamente con la llamada `SYSCTL_STRING`. Luego era recuperado por el módulo consumidor de estadísticas y *parseado* línea a línea para obtener los números de interés.

Una vez implementadas las estructuras específicas para cada tipo de estadística se decidió también implementar un método superador para este objetivo. Mediante `SYSCTL_PROC` se genera un pequeño proceso que publica la dirección de un



puntero a estas estructuras y luego este puede ser obtenido por el módulo que lee las estadísticas. Con esto se evita pasar a *strings* las métricas calculadas contenidas en las estructuras, y a la vez evitamos el *parsear* strings a nivel de *kernel*. El proceso se vuelve más transparente, legible y escalable, a la vez que menos tedioso. Un efecto positivo de estas mejoras fue el ahorro de memoria, algo crítico al estar a nivel de kernel:

- En el módulo `thread_stats` se eliminaron 4 *strings* de 64 bytes, por lo que se ahorraron 256 bytes.
- En el módulo `cpu_stats` se descartaron 3 *strings* de 64 bytes por `CPUSTATES` (5) por la cantidad de núcleos de la CPU del sistema. Aquí se ahorra mucha memoria, ya que por ejemplo en un sistema de sólo 4 núcleos la memoria que ocupaban estos arreglos era de 1280 *bytes*.

4.6.2.5 Módulo `toggle_active_cpu`

Se decidió realizar un mayor desacople de las funciones de este módulo para tener mayor control sobre las mismas. Además, la modularización de funciones aporta mayor legibilidad.

Se dividió la función *callback* que obtenía las estadísticas y apagaba o encendía núcleos en tres *callbacks*: una para obtener las estadísticas, otra para decidir si apagar un core y otra para encenderlos. El nuevo esquema del módulo es el siguiente:

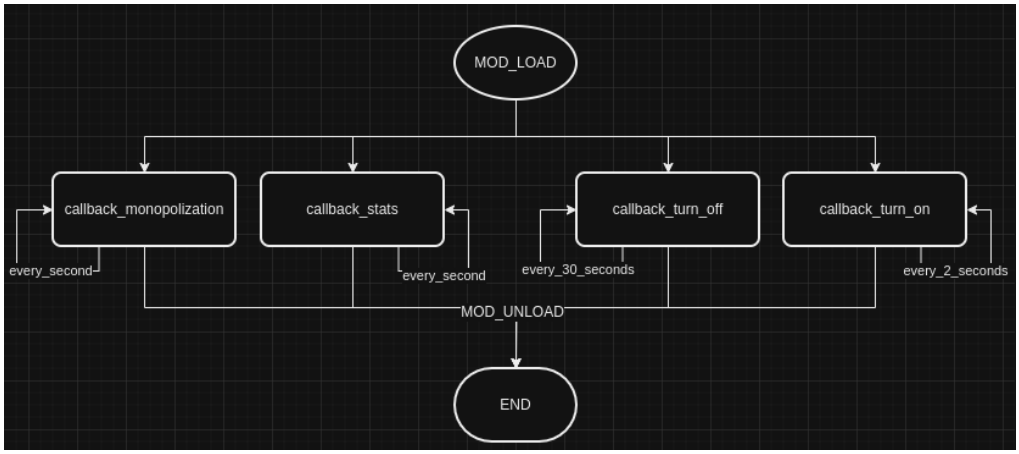


Figura 27: Nuevo esquema de módulo

Y así quedan los diagramas de secuencia de las nuevas funciones *callback*:

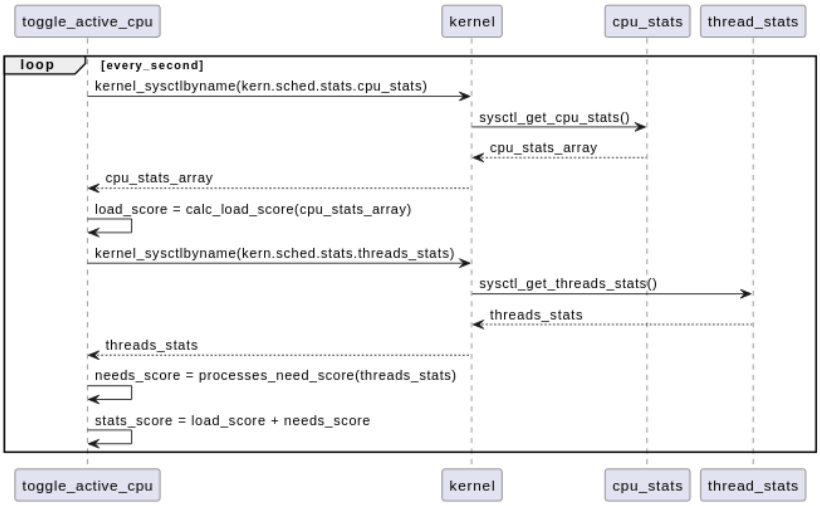


Figura 28: Nueva secuencia de obtención de estadísticas

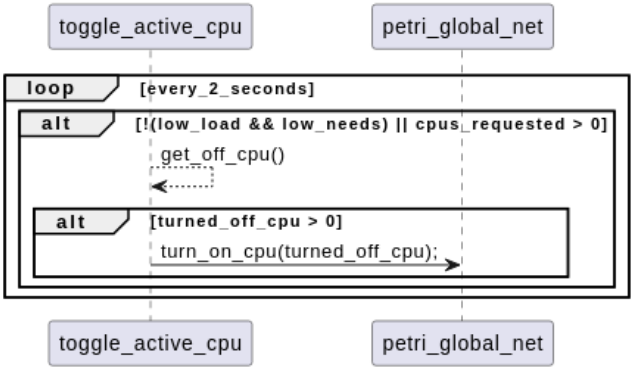


Figura 29: Nueva secuencia de encendido de núcleos

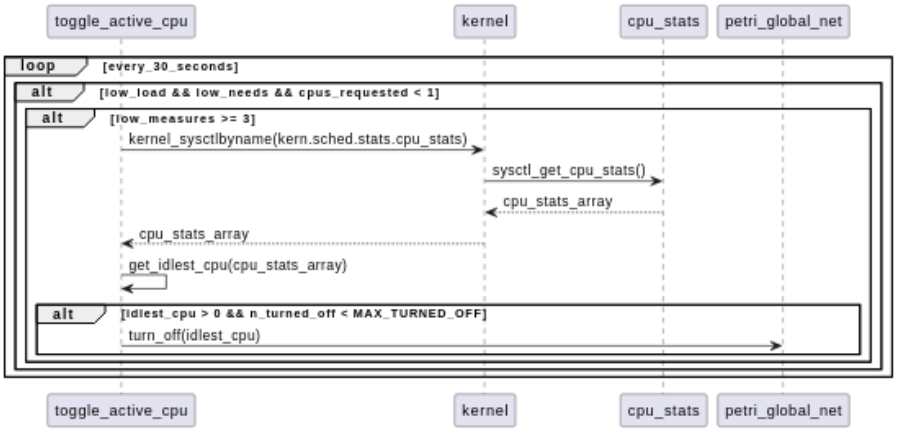


Figura 30: Nueva secuencia de suspendido de núcleos



4.6.2.6 Optimización general

Más allá de la refactorización mencionada, fueron implementadas optimizaciones y correcciones generales revisando instrucciones condicionales, variables innecesarias o inutilizadas, reducción de complejidad cognitiva para mantener funciones más reducidas, extracción de código repetido a funciones reutilizables, definición de constantes útiles, implementación de convenciones y reglas de estilo de código en C e implementación de manejo de errores para evitar escenarios de comportamientos indeterminados, y cambio de clausulas *if–else* simples por operadores ternarios.

4.6.3 Análisis de los resultados

Se presenta, al finalizar esta iteración, una versión mucho más estructurada, optimizada, escalable, refactorizada y funcional del proyecto integrador, con prolijidad y buenas prácticas que nos permitirán trabajar de manera más eficiente en el futuro.

4.6.4 Próximos pasos

Como se mencionó al principio de este trabajo, tomamos como objetivo secundario el mantener este proyecto actualizado a las versiones más recientes del sistema operativo FreeBSD. Con el lanzamiento de la versión 14 de FreeBSD y con la versión 15 ya en desarrollo, el próximo paso es el de la adaptación de lo desarrollado hasta el momento a estas versiones mencionadas, manteniéndonos actualizados y garantizando la compatibilidad total con el sistema operativo en su estado más reciente.



4.7 Séptima iteración

En el transcurso del desarrollo de este trabajo, el sistema operativo FreeBSD estaba en proceso de *release* de su versión 14 y, por consiguiente, en comienzos de desarrollo de la versión 15.

4.7.1 Objetivos

En este punto se decidió actualizar el código de este proyecto y el de los proyectos integrados a las versiones 14 (`stable_release`) y 15 (`current_version` al momento de esta iteración) de FreeBSD. Esto permite dejar el proyecto funcionando en una versión actualizada, lo que facilitaría el interés de otras personas, colaboraciones de otros desarrolladores, y en definitiva lo hace un proyecto más útil y aceptable.

4.7.2 Implementación

Para actualizar el *Proyecto Scheduler* en el estado en el que se encuentra a nuevas versiones del sistema operativo, se comenzó por adaptarlo primero desde FreeBSD 13.2 a la *release* más reciente: FreeBSD 14, lanzada en Noviembre de 2023.

Para esto se instaló una máquina virtual con la imagen oficial de FreeBSD 14, se configuró y se compararon los archivos del *source tree* del *kernel* nuevo con los que modificamos a lo largo del proyecto para agregar/modificar las partes requeridas:

- `conf/files`
- `kern/imgact_elf.c`
- `kern/kern_thread.c`
- `kern/sched_4bsd.c`
- `sys/proc.h`

Además, se agregaron los archivos desarrollados al *source tree*:

- `kern/metadata_elf_reader.c`
- `kern/petri_global_net.c`
- `kern/sched_petri.c`
- `sys/metadata_elf_reader.h`
- `sys/metadata_payloads.h`
- `sys/sched_petri.h`



Una vez listo el *source tree*, se hizo un *build* e *install* del *kernel* nuevo y se procedió a probar el funcionamiento del sistema. Cuando se observó que el *kernel* funcionaba correctamente, se instalaron los módulos de colección de estadísticas y el de manejo del estado de los núcleos de la CPU y se procedió a probarlos mediante los *scripts* desarrollados para este fin.

Para actualizar a FreeBSD 15, se realizó el mismo proceso desde FreeBSD 14 usando una máquina virtual con una imagen *snapshot* del estado actual de la versión *current* de FreeBSD en ese momento (fines de Noviembre de 2023). Cuando se tuvo el nuevo *kernel* en esta versión y se quiso probar, el sistema quedaba colgado en el arranque. Se pensaba que era debido a algún error relacionado a los cambios, hasta que se observó que el proceso de arranque se colgaba siempre en la misma parte, al ejecutarse el *script* `ldconfig` que configura librerías requeridas para la compatibilidad con programas de versiones más antiguas. Por esto se hizo *build* a un nuevo *kernel* sin las opciones de compatibilidad, y así se logró el correcto arranque para observar cómo el sistema en la versión 15 funcionaba también sin problemas. Aquí se puede observar la inestabilidad de una imagen de un sistema operativo en pleno desarrollo y la constante contribución de los desarrolladores, ya que a los días de tener estos problemas observamos cómo se realizó un *commit* que solucionaba el error en el *script* mencionado de arranque.

4.7.3 Análisis de los resultados

Luego de esta iteración, se contaba con la posibilidad de tener el *Proyecto Scheduler* con todo lo implementado a lo largo de este proyecto integrador en 3 versiones diferentes de FreeBSD: 13.2, 14 y 15. Así queda disponible en la última versión posible al momento de esta iteración para evitar la obsolescencia del proyecto y facilitar nuevas implementaciones que se quieran añadir en un futuro.

4.7.4 Próximos pasos

Con el proyecto actualizado para las últimas versiones de FreeBSD disponibles, podemos pasar a adaptar los *plugins* para insertar *metadata* en ejecutables para nuevos tipos de *payloads*.



4.8 Octava iteración

El proyecto *Comunicación desde espacio de usuario a espacio de kernel mediante metadatos en archivos ELF* permite a los procesos darle información sobre ellos al *kernel*. Esto se logra a través de la inserción de *metadata* en sus ejecutables mediante el uso de *plugins* desarrollados para GCC y CLang.

4.8.1 Objetivos

El objetivo de esta iteración es el de modificar los *plugins* mencionados para que acepten distintos tipos de *payloads* y generar uno específico para información sobre uso de CPU de los procesos para proveerle al *scheduler*.

4.8.2 Implementación

Para comenzar, se definió una nueva estructura para el nuevo *payload*. Esta consiste en un booleano que indica si el proceso quiere monopolizar un CPU o no, y un *string* que define los requerimientos de uso de CPU del proceso clasificado en 4 niveles (`LOWPERF`, `STANDARD`, `HIGHPERF` y `CRITICAL`).

```
1 typedef struct {
2   bool monopolize_cpu;
3   sched_proc_type[SCHEDPAYLOAD_STRING_MAXSIZE];
4   padding[3];
5 } Payload_Sched;
```

Sobre el formato de los archivos a insertar como *metadata*, se decidió seguir una convención estilo YAML como se muestra a continuación:

```
1 Sched_Info:
2   Monopolize: false
3   ProcType: "HIGHPERF"
```

Luego se prosiguió con el *plugin* para insertar *metadata* mediante CLang. Para esto hubo que modificar *scripts* de *setup* y *testing*, y el código fuente del *plugin* escrito en C++.

Se modificaron varias secciones del código, siendo las claves las siguientes:

- Detección del tipo de *payload* que se está *parseando*.
- Definición del número de función que se va a insertar. Este número de función es el que le indica al decodificador de *metadata* del *kernel* qué tipo de *metadata* está recibiendo para saber cómo tratarla. Anteriormente se permitía sólo uno y estaba *hardcodeado*.



- Inserción de *metadata* con los distintos tipos de *payload*.
- Control de errores.

A continuación se procedió a realizar lo propio con el *plugin* GCC. Este *plugin* maneja el número de función de los *payloads* mediante el *script* de compilación. Se sigue la misma lógica mencionada para el *plugin* CLang, con la diferencia de que el número de función ya está definido.

Por último, se agregó en el código de `metadata_elf_reader` el tratamiento necesario para decodificar el nuevo *payload* definido.

4.8.3 Análisis de los resultados

Se logró que los *plugins* detecten correctamente los distintos tipos de *payloads* (2 hasta el momento) que se quieren insertar como *metadata* a los ejecutables. Se mantienen los límites de tamaño de la *metadata*, de cantidad de *payloads* a insertar y se restringe a 1 el *payload* `Sched_Info`.

Gracias a esto, el módulo `thread_stats` que colecta información sobre los procesos aprovecha este nuevo *payload* `Sched_Info` para determinar los requerimientos de CPU de los procesos.

A continuación se muestra un ejemplo del resultado de insertar un *payload* `Sched_Info` y tres *payloads* `Binary` mediante el *plugin* Clang:

```
1  User program source code: test.c
2  Plugin argument - 3: schedData/hiperf
3  Plugin argument - 4: dataToInsert/decl_payB.txt
4  Plugin argument - 5: dataToInsert/decl_payA.txt
5  Plugin argument - 6: dataToInsert/decl_payE.txt
6
7  clang14 -Xclang -load -Xclang ./../build/libInsertPayload_Plugin.so  -Xclang -plugin -
       Xclang InsertPayload_Plugin    -Xclang -plugin-arg-InsertPayload_Plugin -Xclang
       schedData/hiperf     -Xclang     -plugin-arg-InsertPayload_Plugin     -Xclang
       dataToInsert/decl_payB.txt   -Xclang   -plugin-arg-InsertPayload_Plugin   -Xclang
       dataToInsert/decl_payA.txt   -Xclang   -plugin-arg-InsertPayload_Plugin   -Xclang
       dataToInsert/decl_payE.txt  -c test.c -I/usr/src/sys -Wno-implicit-int
8  Args[0] = schedData/hiperf
9  Args[1] = dataToInsert/decl_payB.txt
10 Args[2] = dataToInsert/decl_payA.txt
11 Args[3] = dataToInsert/decl_payE.txt
12
13 metadata insertion:
14          static     Metadata_Hdr     metadata_header     __attribute__((__used__,
       __section__(".metadata"))) = {4, sizeof(Payload_Hdr)};
15         static     Payload_Sched     payloads_sched[]     __attribute__((__used__,
       __section__(".metadata"), __aligned__(8))) = {{false,"HIGHPERF"}};
```



```
16          static     Payload_Binary     payloads_bin[]     __attribute__((__used__,
        __section__(".metadata"),              __aligned__(8)))            =
        {{26,52,"5061796C6F61645F422F31362F7A2F772F636861725F73697A65"},
        {24,48,"5061796C6F61645F412F322F342F632F696E745F73697A65"},
        {28,56,"5061796C6F61645F452F31382F35372F6D2F6C6F6E675F73697A65 A"}};
17          static     Payload_Hdr     payload_headers[]     __attribute__((__used__,
        __section__(".metadata")))     =     {{2,     sizeof(payloads_sched[0])},{1,
        sizeof(payloads_bin[0])},{1,             sizeof(payloads_bin[1])},{1,
        sizeof(payloads_bin[2])}}
```

Y aquí lo mismo, pero con GCC:

```
1  Plugin argument - 2: test/schedData/hiperf
2  Plugin argument - 3: test/dataToInsert/decl_payA.txt
3  Plugin argument - 4: test/dataToInsert/decl_payB.txt
4  Plugin argument - 5: test/dataToInsert/decl_payC.txt
5
6   make test PLUGIN_ARGS='-fplugin-arg-InsertPayload_GCCPlugin-2=test/schedData/hiperf -
        fplugin-arg-InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payA.txt    -fplugin-
        arg-InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payB.txt        -fplugin-arg-
        InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payC.txt '
7
8    gcc  -o  metadataTestProgramGCC  test/test.c  -fplugin=./InsertPayload_GCCPlugin.so  -
        fplugin-arg-InsertPayload_GCCPlugin-2=test/schedData/hiperf          -fplugin-arg-
        InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payA.txt            -fplugin-arg-
        InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payB.txt            -fplugin-arg-
        InsertPayload_GCCPlugin-1=test/dataToInsert/decl_payC.txt  -I/usr/src/sys
9
10 Number of arguments of this plugin: 4
11 Arg [0]: - Key: [2] - Value: [test/schedData/hiperf]
12 Arg [1]: - Key: [1] - Value: [test/dataToInsert/decl_payA.txt]
13 Arg [2]: - Key: [1] - Value: [test/dataToInsert/decl_payB.txt]
14 Arg [3]: - Key: [1] - Value: [test/dataToInsert/decl_payC.txt]
15
16 To be replaced:
17      __PAYLOAD__=static   Metadata_Hdr   metadata_header   __attribute__((__used__,
        __section__(".metadata"))) = {4, sizeof(Payload_Hdr)};
18          static     Payload_Sched     payloads_sched[]     __attribute__((__used__,
        __section__(".metadata"), __aligned__(8))) = {{false,"HIGHPERF"}};
19          static     Payload_Binary     payloads_bin[]     __attribute__((__used__,
        __section__(".metadata"),              __aligned__(8)))            =
        {{24,48,"5061796C6F61645F412F322F342F632F696E745F73697A65"},
        {26,52,"5061796C6F61645F422F31362F7A2F772F636861725F73697A65"},
        {28,56,"5061796C6F61645F432F362F31302F652F666C6F61745F73697A650A"}};
20          static     Payload_Hdr     payload_headers[]     __attribute__((__used__,
        __section__(".metadata")))     =     {{2,     sizeof(payloads_sched[0])},{1,
        sizeof(payloads_bin[0])},{1,             sizeof(payloads_bin[1])},{1,
        sizeof(payloads_bin[2])}};
```



4.8.4 Próximos pasos

Llegado este punto del proyecto, se decidió revisar y realizar una retrospección sobre todo el trabajo realizado concluyendo que el próximo paso es el de intentar simplificar la red de Petri que modela el *scheduler*.



4.9 Novena iteración

Las modificaciones realizadas durante el transcurso de este proyecto sobre la red de Petri con la que se comenzó provocaron un aumento de complejidad de la misma, lo que puede ser un problema a la hora de escalarla a un sistema multiprocesador.

4.9.1 Objetivos

En esta iteración se tiene como objetivo simplificar la red de Petri de recursos del *scheduler* en la medida de lo posible para mejorar su análisis e implementación en el sistema.

4.9.2 Implementación

Al detenerse a analizar nuevamente la red de Petri en uso, se notó que la solución propuesta en la Cuarta iteración sobre el equilibrado de colas cuando hay núcleos suspendidos estaba incompleta. Esto resultó ser así porque la plaza que representa el estado suspendido (`QUEUED_OR_SUSP`) sensibiliza el disparo de la transición de equilibrio (`THROW`) y el *token* no vuelve a la plaza luego de este disparo. Para salvar este problema se modificaron algunas incidencias, se eliminó la plaza `PLACE_CTRL` que limitaba innecesariamente la capacidad de dicha plaza, y cuando se dispara la transición `THROW` se vuelve a llenar la plaza de las CPU que lo necesitan (suspendidas) por código. Este comportamiento no fue detectado anteriormente ya que todo seguía funcionando, sólo que la red no capturaba el comportamiento real del *scheduler* ya que, según su punto de vista, todos los hilos que se ejecutaban provenían de la plaza de encolado global. Estos cambios pueden apreciarse a continuación:

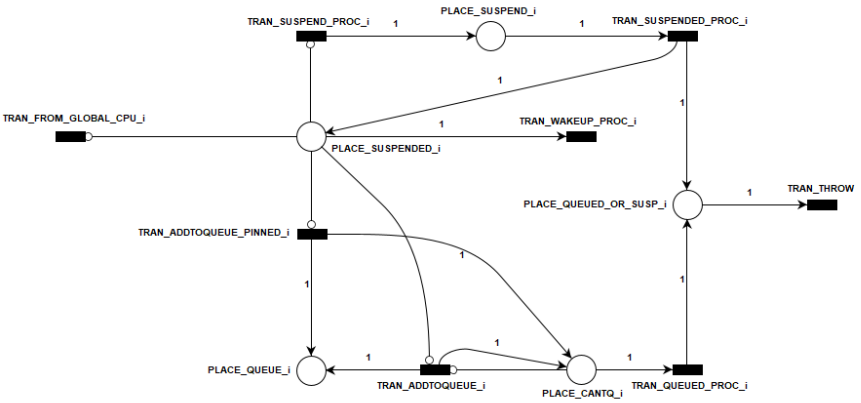


Figura 31: Red de petri corregida

En el anexo A.4 se puede observar la secuencia en una red simplificada: cuando un núcleo va a ser suspendido, se dispara la transición `TRAN_SUSPEND_PROC`, lo que dispara automáticamente `TRAN_SUSPENDED_PROC`, lo cual marca al núcleo como suspendido y llena la plaza que representa el estado *suspendido o encolado*. Esto permite que, cuando sea necesario, la transición `THROW` cumpla su cometido, pero sólo la primera vez que esto ocurre. Puede notarse también que aplicar estas modificaciones a la red permite que la transición `THROW` realice su función no sólo una vez, sino siempre.

A continuación se empezaron a investigar nuevos caminos para tratar de reducir la complejidad del modelo:

1. Partiendo del esquema original de suspendido/encendido de núcleos en la red obtenido en la segunda iteración (Figura 12: Red de Petri con suspensión de núcleo) se exploraron dos posibilidades:

   - Agregar una transición `ADDTOQUEUE_SUSP` que se dispara cuando le correspondería encolar a un núcleo suspendido. Esto sólo agrega un *token* en la plaza `CANTQ` para que la transición global `THROW` pueda dispararse y equilibrar el encolado *round robin:*

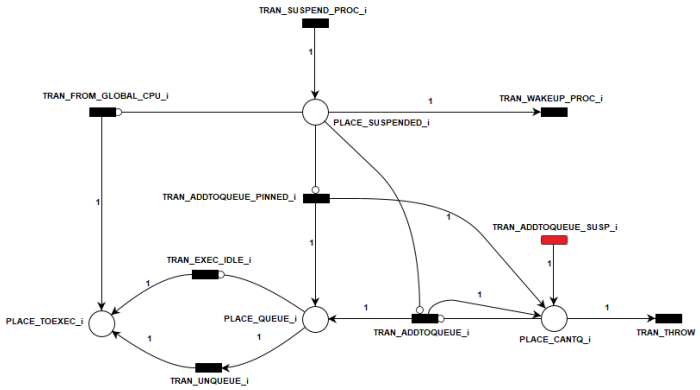


Figura 32: Alternativa de suspensión simplificada (1)

- Agregar una nueva plaza (`TO_SUSPEND`) y 2 transiciones (`TO_SUSPEND` y `RE_SUSPEND`). Cuando se suspende un núcleo, se agrega un *token* en `TO_SUSPEND` y esto dispara la transición automática `TO_SUSPEND` que agrega un *token* en `CANTQ` (maneja la inhibición de encolado, y `THROW` la utiliza para equilibrar el encolado del sistema) y otro en la plaza `SUSPEND` (inhibe el encolado global). Cuando se dispara `THROW` para equilibrar el encolado, mediante la transición automática `RE_SUSPEND` se reinicia este proceso para reflejar el estado suspendido del core y no afectar el método *round robin* de la red para asignar hilos a las colas de los núcleos:

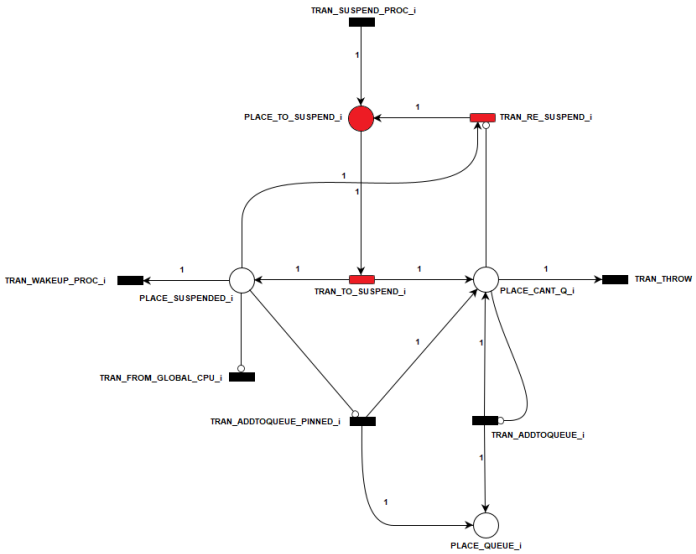


Figura 33: Alternativa de suspensión simplificada (2)

2. Al solucionar los *kernel panics* cuando se intentaba utilizar el algoritmo de afinidad de hilos en núcleos del *scheduler* 4BSD mediante la transición `ADDTOQUEUE_PINNED` (*Segunda iteración*), concluimos que el esquema para equilibrar colas mediante `THROW` pierde su sentido. Es por esto que se decidió modificar la red quitando las partes que surgen por dicho esquema, lo cual también llevó a reducir y simplificar bastante el código resultante:

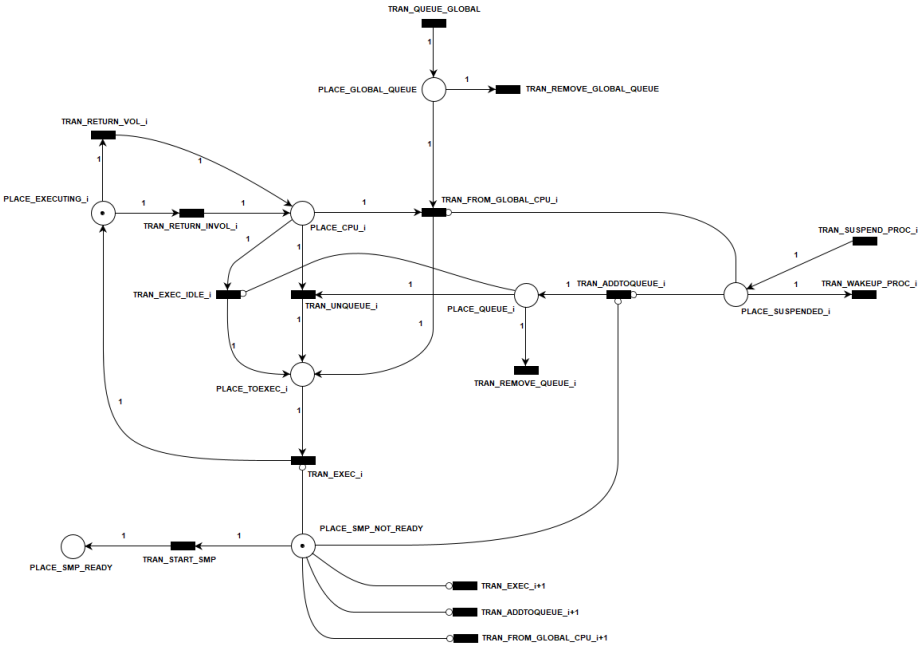


Figura 34: Red simplificada para un núcleo

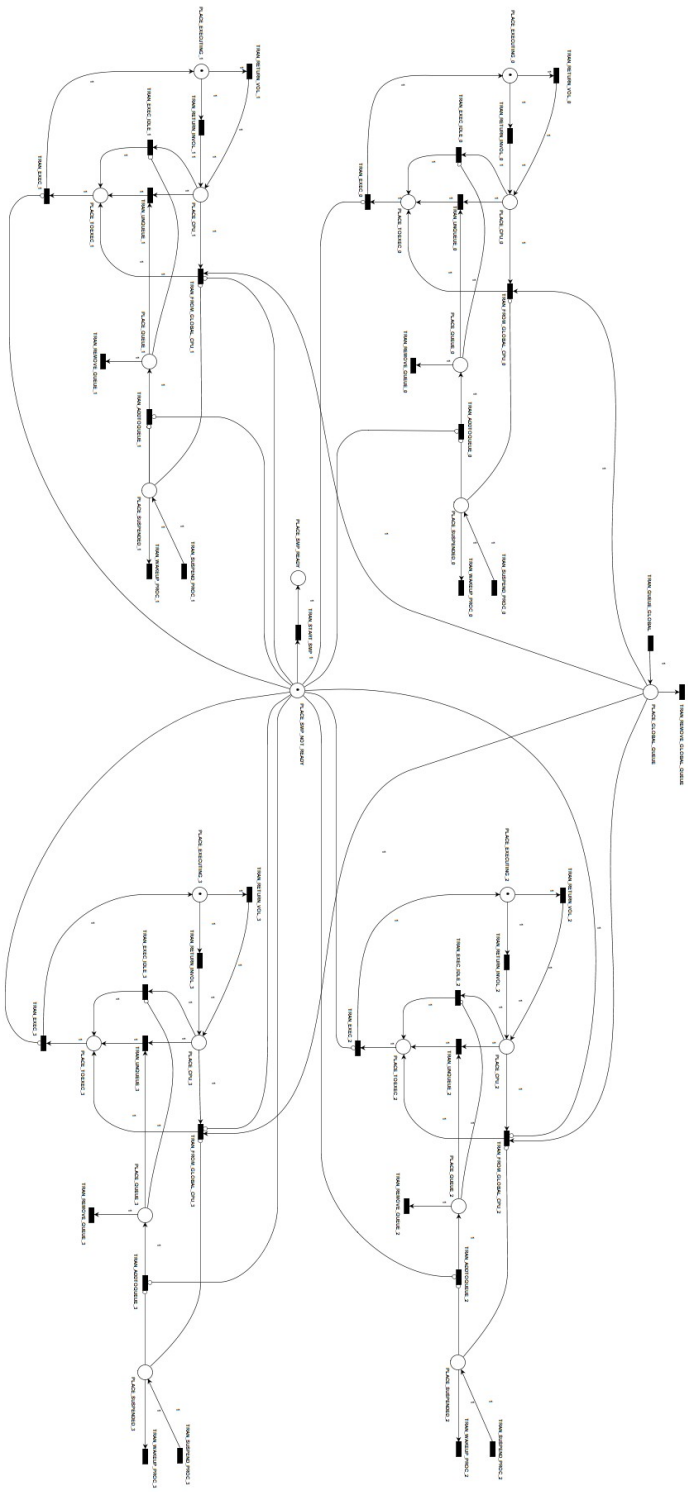


Figura 35: Red simplificada para un sistema monoprocesador de 4 núcleos



4.9.3 Análisis de los resultados

Se comprobó que cada red propuesta siga cumpliendo con las propiedades de ser pura y libre de *deadlocks*, además de ser inseguras y no acotadas por las plazas de encolado. Se realizaron las modificaciones pertinentes en el código y se realizaron pruebas corroborando que todo funcione correctamente y se analizaron las diferencias entre ellas.

Se observa que existen dos redes sobre las que se debe decidir para utilizar en el proyecto según el rumbo que se decida tomar:

- Si es preferible que la red tenga control sobre la asignación de hilos a colas de núcleos, tal vez lo mejor sería utilizar la red propuesta en la figura 32, ya que es la que aporta menor complejidad obteniendo resultados similares.
- En el caso de ir por una red que permita escalar mejor a casos de computadoras con múltiples CPUs para aportar funcionalidades sobre el estado de los mismos sin tomar decisiones en asignación de hilos a colas, la red propuesta en la figura 33 es óptima debido a su simpleza. Esto es fácilmente observable en los cambios sobre el modelo de la red de recursos y el código que se listan en el siguiente inciso.
- Red de Petri de recursos:
  - Reducción del número de plazas de 9 a 5, de transiciones de 15 a 10 y de transiciones jerárquicas de 9 a 6.
  - Eliminación de transiciones automáticas.
  - Eliminación de la transición global `THROW`.
- Gracias a las simplificaciones logradas en esta iteración, se consiguió reducir el código de *kernel* involucrado para modelar el *scheduler* con la red.

4.9.4 Próximos pasos

Luego de realizar un análisis poniendo en la ecuación los objetivos planteados inicialmente, el estado actual del proyecto y el tiempo de desarrollo transcurrido, se decidió que el próximo paso sería establecer el estado final del sistema.



4.10 Décima iteración

Luego de debatir en equipo sobre los objetivos planteados, el progreso realizado en el tiempo y el esfuerzo que implicarían posibles futuras iteraciones, se tomó la decisión de que se desarrollaría una última iteración para determinar un estado final de este proyecto y así quien continúe pueda tomarlo como punto de partida.

4.10.1 Objetivos

El objetivo de esta iteración es el de obtener y presentar la versión final del proyecto desarrollado. En este repositorio se puede observar la historia completa del trabajo desarrollado: `https://github.com/frandaniele/pi_doc`.

4.10.2 Implementación

Para presentar el modelo final completo, se repasaron las tareas desarrolladas y se realizaron las últimos pruebas para verificar que el sistema funcione como debería en todas las versiones. Así fue que encontramos dos *bugs* que nos encargamos de eliminar:

- El módulo de gestión de estado de los núcleos (`toggle_active_cpu`) no hacía correctamente el *release* de un núcleo monopolizado cuando quien lo monopolizaba terminaba de ejecutarse (*commit*).
- El *plugin* de GCC para insertar *metadata* en ejecutables ELF no funcionaba correctamente, insertando sólo el *header* de *metadata*, omitiendo el *header* de *payloads* y los *payloads* en sí (*commit*).

4.10.3 Estado final

4.10.3.1 Versiones de FreeBSD

Uno de los objetivos de este proyecto era actualizar el código de los trabajos anteriores a versiones más recientes de FreeBSD. Eso fue realizado partiendo de una parte desarrollada en FreeBSD 11, otra en FreeBSD 12 y otra también en FreeBSD 13, llevándose primero a FreeBSD 13.2 que era la última versión estable al momento del inicio de este proyecto integrador. Luego se llevó a FreeBSD 14, que fue la última versión estable de FreeBSD lanzada durante el desarrollo del proyecto, y finalmente se

adaptó a FreeBSD 15 estando todavía en etapa de desarrollo al mes de Febrero de 2024.

En este *fork* del código fuente de FreeBSD se pueden ver los cambios que se realizaron al *source tree* del sistema operativo en las ramas de las versiones sobre las que se trabajó: `13`, `14` y `main` (`15-CURRENT`).

Se provee también documentación para poder aprovisionar automáticamente una máquina virtual en la versión deseada con el entorno generado por el proyecto. Esta documentación está detallada en el anexo B.

4.10.3.2 Red de Petri: *Scheduler* 4BSD

- ***Modelo de red de Petri para el comportamiento de un hilo***: Este modelo queda sin modificaciones respecto al original del proyecto de los autores Papp y Turina ya que representa correctamente los distintos estados que puede tener un hilo:

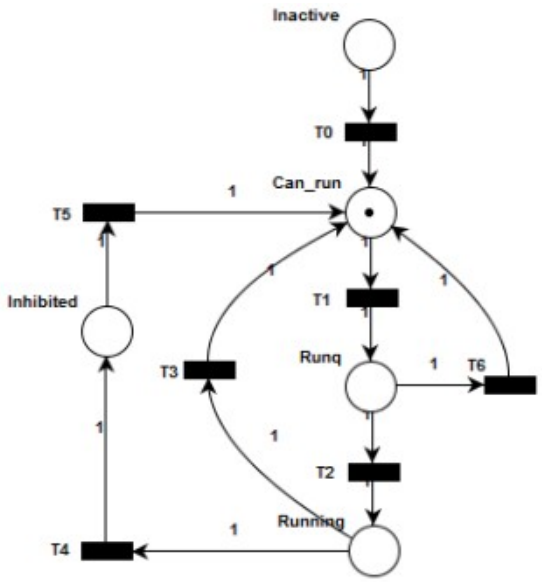


Figura 36: Red de Petri para los estados de un hilo

- ***Modelo de la red de Petri de recursos***: Luego de las diversas modificaciones sobre la red original del proyecto de los autores Papp y Turina realizadas tanto en el proyecto de Drudi y Goldman como en el presente, se decidió utilizar la red obtenida en la iteración 9 (figuras 34 y 35).



- ***Jerarquía de transiciones entre modelo del hilo y red de recursos***:
  - Transiciones `ADDTOQUEUE`, `EXEC_IDLE` y `QUEUE_GLOBAL` de la red de recursos son jerárquicas a la transición `ON_QUEUE` del hilo, la cual lo pasa de estado `CAN_RUN` a `RUNQ`.
  - La transición `EXEC` de la red de recursos es jerárquica a `SET_RUNNING`, que pasa el hilo de `RUNQ` a `RUNNING`.
  - `REMOVE_QUEUE` y `REMOVE_GLOBAL_QUEUE` de la red de recursos son jerárquicas a `REMOVE` del modelo de hilo, provocando el paso de estado `RUNQ` a `CAN_RUN`.
  - `RETURN_INVOL` de la red de recursos es jerárquica con `SWITCH_OUT`, que provoca el cambio de estado `RUNNING` a `CAN_RUN` de los hilos.
  - `RETURN_VOL` de los recursos es jerárquica a `TO_WAIT_CHANNEL` que pasa al hilo de `RUNNING` a `INHIBITED`.

4.10.3.3 *Metadata* en ejecutables ELF

Al proyecto que inició la posibilidad de introducir información desde espacio de usuario al de *kernel* a través de *metadata* en ejecutables ELF [2], se le hicieron las modificaciones necesarias para poder insertar un nuevo tipo de *payload* con información de *scheduling* sobre los procesos. Este nuevo *payload* sigue el siguiente formato YAML:

```yaml
Sched_Info:
    Monopolize: bool
    ProcType: string
```

Siendo `ProcType` uno de los siguientes: `LOWPERF`, `STANDARD`, `HIGHPERF` o `CRITICAL`. Si no se cumple con el formato, este no será detectado como *payload* ed tipo `Sched_Info` y será tratado como *payload* `Binary`.

Para detectar este nuevo *payload* y *parsearlo* en ejecución, se debieron modificar los archivos de *kernel* `metadata_reader.c`, `metadata_reader.h` y `metadata_payloads.h`, además de los dos *plugins* existentes para insertar la *metadata* en los ejecutables: uno utilizando GCC y otro utilizando CLang.

Estos *plugins* fueron actualizados para ser compatibles hasta las versiones 12.3 de GCC y 14 de CLang. El *plugin* de CLang es compatible además con la versión 16 del mismo, pero sólo si se utiliza junto con la versión 14 de LLVM ya que la versión 16 de este último causaba problemas de compilación que no pudieron ser solucionados.



4.10.3.4 *Scheduling* por tipo de proceso

Aprovechando las posibilidades que proveen las funcionalidades aportadas por los proyectos integrados en adición a lo desarrollado en el presente trabajo, se desarrollaron una serie de módulos de *kernel* buscando gestionar el estado de los núcleos del sistema operativo a partir de estadísticas e información recolectada. Estos módulos son:

- cpu_stats: Obtiene y transmite estadísticas sobre el uso de los núcleos de la CPU.
- thread_stats: Obtiene y transmite estadísticas a nivel de hilos y procesos, además de información de los procesos a través de *metadata* en ejecutables.
- toggle_active_cpu: Gestiona el estado de los cores del sistema.

Los módulos comparten esta pequeña librería y existen dos formas para cargarlos en *kernel*:

- Compilación y carga dinámica sirviéndose de *makefiles* genéricos provistos por FreeBSD.
- Compilación y carga en *booteo* al incorporarlos al *source tree* y modificando los archivos del sistema necesarios. Esto se puede observar en el repositorio *fork* del código fuente de FreeBSD con las modificaciones propias.

4.10.4 Análisis de los resultados

Esta iteración marca un punto importante en el proyecto integrador. Los resultados revelan el logro de los objetivos establecidos, incluida la presentación de la versión final del trabajo. Durante esta fase, se observa que el modelo de planificador propuesto ha evolucionado teniendo en cuenta múltiples aspectos y ha sido implementado con éxito en el sistema operativo FreeBSD, integrando componentes claves para la gestión de procesos y núcleos de una CPU. Se ha logrado actualizar el código a versiones más recientes de FreeBSD, garantizando la compatibilidad y funcionalidad del proyecto en entornos operativos actuales, y manteniendo buenas prácticas de programación. Los cambios realizados en la red de Petri y la implementación de *metadata* han mejorado la capacidad del sistema para gestionar eficientemente los recursos del sistema, mientras que los módulos de *kernel* desarrollados han demostrado ser efectivos en la gestión del estado de los núcleos de la CPU. En resumen, el análisis de los resultados confirma el éxito del proyecto en la



consecución de sus objetivos, destacando la relevancia y viabilidad de las soluciones propuestas en el contexto de la planificación de recursos en sistemas operativos.

4.10.5 Próximos pasos

Con la conclusión de la décima iteración, se marca el cierre de este trabajo, pero también se abre la puerta a futuras investigaciones y desarrollos. En este punto se evaluarán detalladamente los trabajos futuros a abordar, que servirán como continuación y ampliación de este *Proyecto Scheduler*, aprovechando la experiencia y los recursos acumulados durante el desarrollo de este proyecto integrador.



5. Síntesis de los resultados obtenidos

Los resultados obtenidos se centran en la incorporación y el desarrollo de diversas funcionalidades dentro del sistema operativo FreeBSD, tales como la suspensión y monopolización de núcleos de la CPU, la incorporación de *metadata* en los *headers* de ejecutables ELF para compartir información de necesidad de procesamiento al *scheduler*, la recopilación de estadísticas y el proceso de toma de decisiones en base a las mismas. El desarrollo se hizo partiendo del modelo base del *scheduler* 4BSD [1] utilizado en el *Proyecto Scheduler*.

5.1 Suspensión de núcleos

Una de las principales funcionalidades del proyecto es la capacidad de suspender núcleos inactivos de la CPU [3]. Este mecanismo permite al *scheduler* identificar núcleos que no están siendo utilizados y suspenderlos temporalmente. Para esto, se realiza una evaluación periódica de la carga de trabajo del sistema, permitiendo que los núcleos suspendidos se reactiven cuando la demanda de procesamiento aumenta.

5.2 Monopolización de núcleos

La monopolización [3] es un componente clave en este proyecto. Esta funcionalidad permite que ciertos procesos críticos soliciten el uso exclusivo de uno de los núcleos disponibles de la CPU, característica especialmente útil en aplicaciones que requieren un procesamiento constante y costoso, logrando aprovechar el poder de procesamiento de un núcleo completo.

5.3 Incorporación de *metadata* en *headers* de ejecutables ELF

La *metadata* personalizada en los ejecutables ELF [2] es el mecanismo de comunicación utilizado, en este proyecto, por un proceso para proveer información detallada al *kernel* sobre sus requisitos de procesamiento, lo cual permite al *scheduler* tomar decisiones que contemplen estas necesidades. Es en esta sección del *header* de



estos ejecutables donde se inserta la información sobre necesidades de procesamiento y/o monopolización.

5.4 Recopilación de estadísticas

La recopilación de estadísticas ha sido fundamental para evaluar el funcionamiento de las nuevas funcionalidades implementadas. Se han desarrollado módulos de *kernel* específicos para monitorizar y registrar estadísticas de procesamiento de los núcleos de la CPU para evidenciar tanto el balance de carga por parte del *scheduler* como el funcionamiento de los mecanismos de suspensión y monopolización de los núcleos de la CPU.

5.5 Toma de decisiones

Finalmente, se propuso e implementó una política de toma de decisiones en el *scheduler* utilizando la información obtenida de la *metadata* de los ejecutables y las estadísticas de la carga de la CPU. De esta manera, el *scheduler* puede tomar decisiones dinámicamente en base al estado del sistema; esto incluye la capacidad de distribuir la carga de trabajo entre los núcleos de la CPU teniendo en cuenta sus necesidades y reaccionar rápidamente a cambios en la demanda de recursos.

5.6 Corolario

Si bien los resultados obtenidos en este proyecto integrador son alentadores, es importante tener en cuenta que este trabajo es una prueba de concepto. No se ha profundizado en rigurosidades teóricas ni se han realizado evaluaciones exhaustivas de rendimiento, ya que el objetivo principal ha sido un primer acercamiento a la unificación de los trabajos que componen el *Proyecto Scheduler*. Por lo tanto, futuros estudios y desarrollos serán necesarios para validar y optimizar las soluciones propuestas en escenarios más específicos.



6. Trabajos futuros

A pesar de los avances significativos logrados en el desarrollo del mecanismo de *scheduling*, existen diversas áreas que requieren atención adicional y exploración en futuras etapas del *Proyecto Scheduler*. En esta sección, se identifican y discuten varios trabajos futuros que se proponen como extensiones del proyecto integrador presentado. Estos trabajos abarcan desde mejoras técnicas específicas hasta consideraciones más amplias sobre la escalabilidad del proyecto.

- ***Implementación de apagado de núcleos por hardware***: Una dirección prometedora para la optimización del consumo de energía y la gestión de recursos en sistemas embebidos sería la posibilidad de apagar físicamente los núcleos de la CPU cuando no estén en uso. Explorar esta opción requerirá investigaciones adicionales sobre la viabilidad técnica y las implicaciones de diseño.
- ***Mejora del algoritmo de toma de decisiones sobre el estado de los núcleos***: El refinamiento continuo del algoritmo utilizado para determinar el estado de los núcleos de la CPU es esencial para optimizar el rendimiento y la eficiencia del sistema.
- ***Evaluación de distintas redes de Petri***: La evaluación de diferentes modelos de redes de Petri basados en el modelo base del *scheduler* 4BSD puede proporcionar información valiosa sobre comparaciones de *performance* para determinar cuál de las variantes ofrece el mejor rendimiento en términos de planificación de tareas y asignación de recursos.
- ***Incorporación de módulos de kernel desarrollados al sistema***: Integrar los módulos de *kernel* desarrollados en el marco de este proyecto en el sistema operativo FreeBSD es un paso crucial para su adopción y utilización práctica. Este proceso requerirá pruebas exhaustivas, optimización de rendimiento y garantía de compatibilidad con versiones futuras del sistema operativo.
- ***Escalado del proyecto a sistemas multiprocesador multinúcleo***: Con miras hacia el futuro, escalar el proyecto para que sea compatible con sistemas multiprocesador multinúcleo es un objetivo no menor. Este trabajo implica la adaptación del sistema para aprovechar al máximo la capacidad de procesamiento paralelo, garantizando una distribución equitativa de la carga de trabajo y minimizando las congestiones en el sistema.



En resumen, los trabajos futuros identificados en este proyecto integrador ofrecen una hoja de ruta clara para la evolución y el perfeccionamiento continuo del *Proyecto Scheduler* del cual se es parte.



7. Conclusiones

Durante el transcurso de este proyecto integrador, hemos tenido el privilegio de colaborar estrechamente con el *Proyecto Scheduler*, liderado por los ingenieros Maximiliano Eschoyez y Nicolás Papp, en asociación con el Laboratorio de Arquitectura de Computadoras (LAC), quienes nos confiaron el desafío de proponer un modelo de planificador que unificara los trabajos previos y contemporáneos al mismo marco de trabajo. Nuestro objetivo más específico era aprovechar las capacidades emergentes de decisión sobre el estado de los núcleos de la CPU, basadas en información detallada proporcionada por los procesos en ejecución.

Además de contribuir al objetivo general del mencionado proyecto, estamos en condiciones de decir que pudimos lograr los objetivos concretos que definimos para nuestro trabajo:

- La integración de los trabajos previos,
- La actualización del sistema a la última versión disponible de FreeBSD, y finalmente
- La obtención de un módulo que permita tomar decisiones de planificación basado en información de los procesos.

Cabe mencionar que este proyecto nació principalmente como una prueba de concepto, por lo que el fin principal era determinar la viabilidad de los objetivos propuestos, mas no el testeo extensivo, el rigor científico ni la búsqueda de la *performance* o eficiencia máxima. Estos son objetivos que habrían extendido este trabajo excesivamente en el tiempo.

Aunque reconocemos que nuestro trabajo representa un primer paso significativo hacia una solución más completa, estamos orgullosos de haber establecido una base sólida para futuras investigaciones y desarrollos en el ámbito de la planificación de recursos en sistemas operativos. Al integrar los diversos componentes del Proyecto Scheduler en un entorno de trabajo funcional y cohesivo, hemos allanado el camino para investigaciones posteriores que podrían profundizar en nuestra comprensión y mejorar la eficiencia de los sistemas informáticos.

Con la entrega de guías de configuración detalladas y el entorno de trabajo funcional proporcionado, aspiramos a facilitar el inicio de futuros proyectos que puedan aprovechar y expandir nuestra investigación tomando como base la sección de trabajos futuros.



En última instancia, este proyecto no solo representó un punto significativo en nuestra trayectoria académica, sino que también sirvió como un testimonio del entendimiento y aplicación de los conocimientos adquiridos a lo largo de nuestra formación en la carrera. Desde los fundamentos teóricos de la planificación de procesos hasta la implementación práctica en el sistema operativo FreeBSD, hemos sido capaces de fusionar y aplicar los conceptos y habilidades aprendidos en una variedad de asignaturas, incluyendo sistemas operativos, redes de computadoras, algoritmos y estructuras de datos, entre otras. Este proyecto ha reforzado nuestra comprensión de cómo los principios teóricos se traducen en soluciones concretas y funcionales en el mundo real, y nos ha equipado con las habilidades y la confianza necesarias para abordar desafíos futuros con determinación y éxito.



8. Bibliografía

[1] Papp, Nicolás; Turina, Tomás: *Modelado del planificador a corto plazo con redes de Petri*; 2017

[2] Cancinos, José María; González Julián: *Comunicación desde espacio de usuario a espacio de kernel mediante metadatos en archivos ELF*; 2022

[3] Drudi, Leandro; Goldman, Nicolás: *Optimización del planificador a corto plazo con redes de Petri*; 2024

[4] McKusick, Marshall; Neville-Neil, George: *Thread scheduling in FreeBSD 5.2*; 2004

[5] McKusick, Marshall; Neville-Neil, George: *The design and implementation of the FreeBSD operating system (2nd edition)*; 2004

A. Anexo: Secuencias de disparos en la red de Petri

En el siguiente anexo se muestran diversas situaciones respecto a las secuencias de disparos referidas a la función de la transición `THROW`, en una red simplificada de 4 núcleos teniendo en cuenta sólo las plazas y transiciones de encolado.

A.1 Transición `THROW`

En esta secuencia se puede observar cómo cumple su función la transición global `THROW`. Al inicio, todos los núcleos están en condiciones de encolar ya que su transición `ADDTOQUEUE` está sensibilizada. Luego una vez que uno encola, su transición `ADDTOQUEUE` está inhibida, y una vez que todos los núcleos del sistema encolaron la transición `THROW` está lista para cumplir su función. Al dispararse esta transición luego de que las colas estén equilibradas, se vuelve a habilitar el encolado de todos los núcleos.

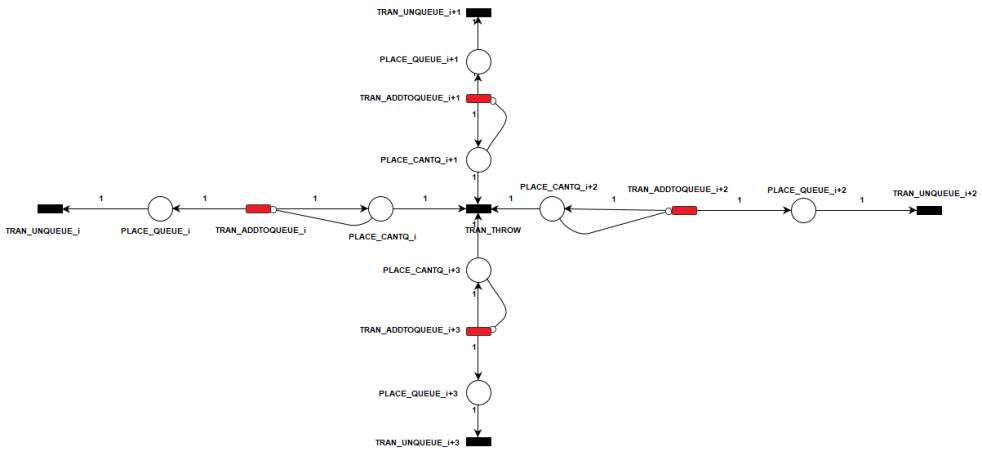


Figura 37: Secuencia de transición `THROW` (1)

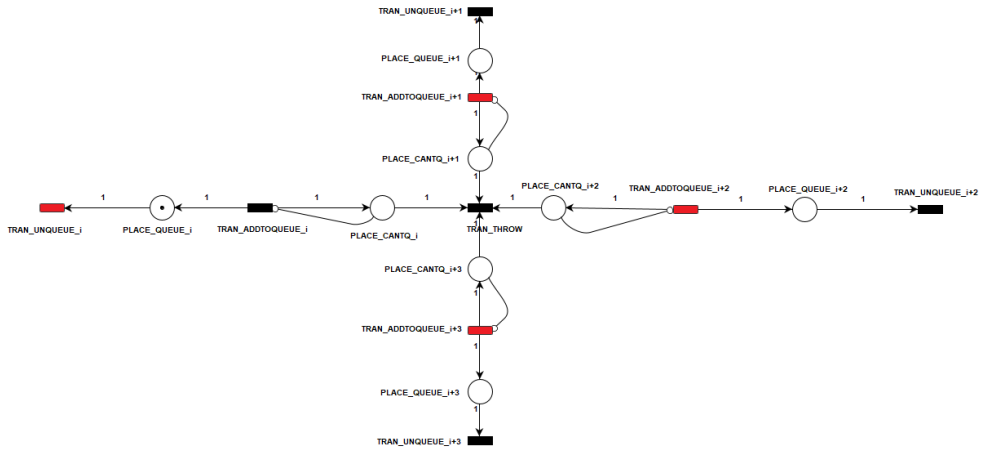


Figura 38: Secuencia de transición `THROW` (2)

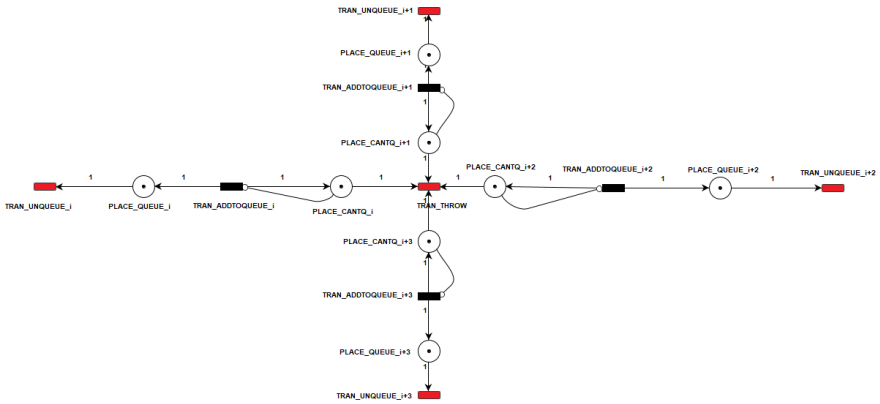


Figura 39: Secuencia de transición `THROW` (3)

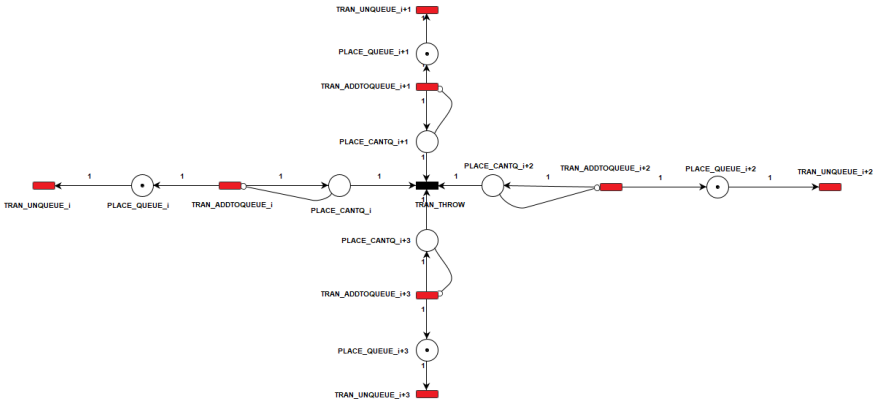


Figura 40: Secuencia de transición `THROW` (4)

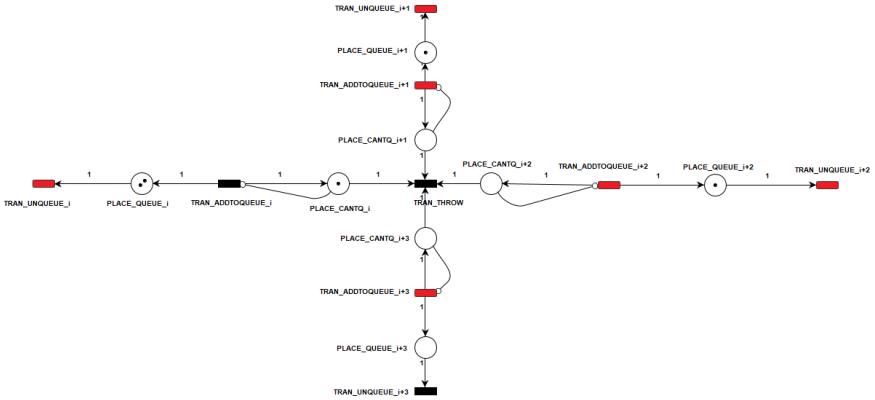


Figura 41: Secuencia de transición `THROW` (5)

A.2 Suspensión y THROW

A continuación se procede a mostrar el problema que trae aparejado la función de suspendido y encendido de núcleos para el equilibrado de colas con la transición `THROW`. Observamos cómo cuándo un núcleo está suspendido no puede encolar y el resto sí. Una vez que los demás encolaron hilos, llega un punto en que no pueden más debido a que están esperando que `THROW` sea disparada para volver a habilitar las colas, pero esto no será posible ya que al haber un núcleo suspendido, éste no llenará su plaza `CANTQ` y `THROW` no podrá ser habilitada.

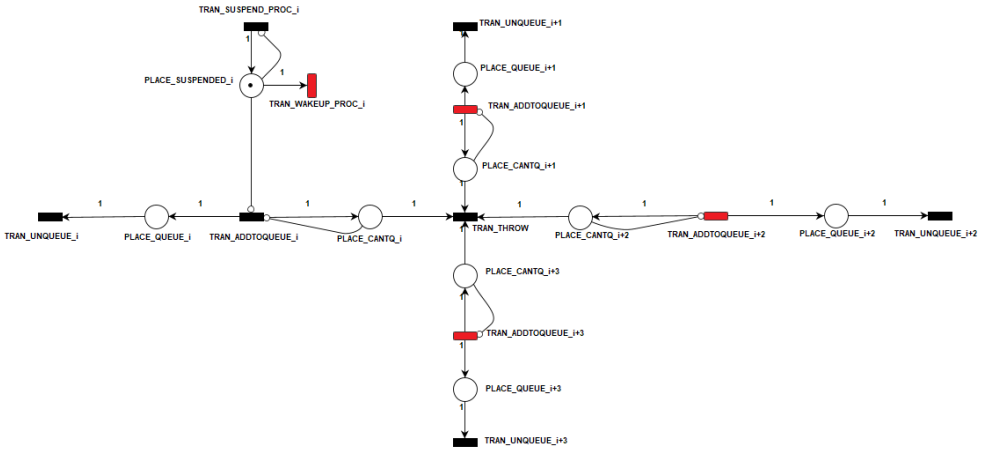


Figura 42: Secuencia de suspensión y `THROW` (1)



Figura 43: Secuencia de suspensión y `THROW` (2)

A.3 Suspensión con nuevo `THROW`

Aquí vemos la secuencia de la solución propuesta para que la función de `THROW` tenga en cuenta cuándo hay un núcleo suspendido. Primero se ve cómo iteran las transiciones de suspendido y ponen al núcleo en el estado suspendido, a la vez que se llena la nueva plaza que determina si un núcleo encoló o está suspendido para que `THROW` pueda ser sensibilizada en cualquier caso, lo cual se observa a continuación cuando los núcleos restantes encolaron hilos.

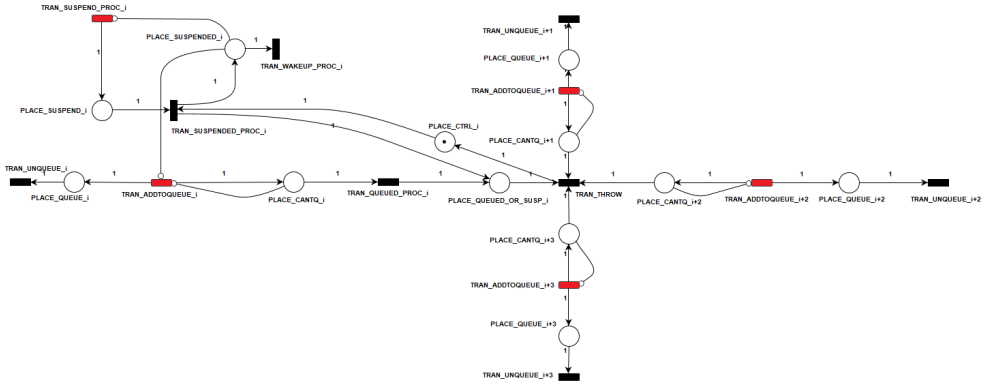


Figura 44: Secuencia de suspensión con nuevo `THROW` (1)

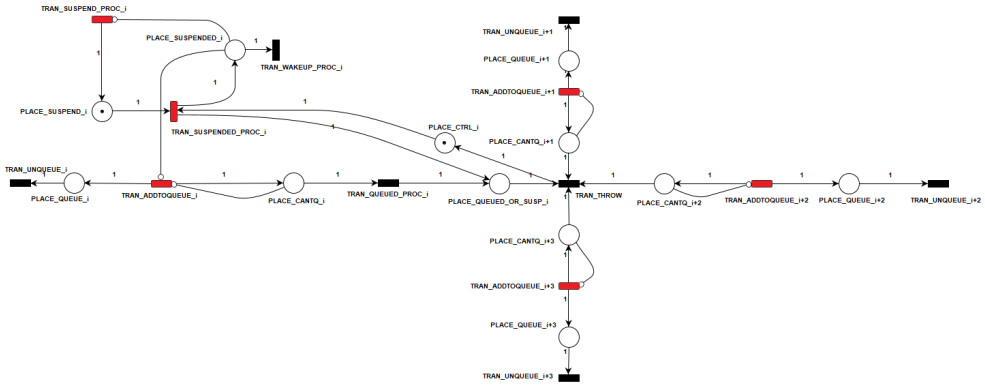


Figura 45: Secuencia de suspensión con nuevo `THROW` (2)

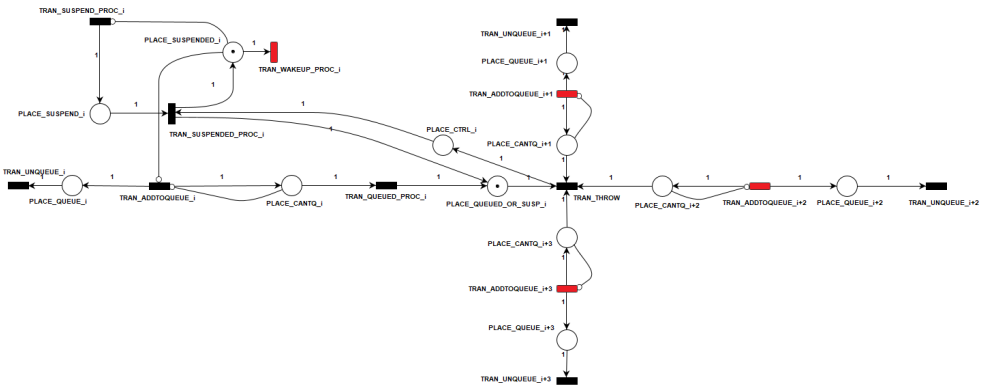


Figura 46: Secuencia de suspensión con nuevo `THROW` (3)

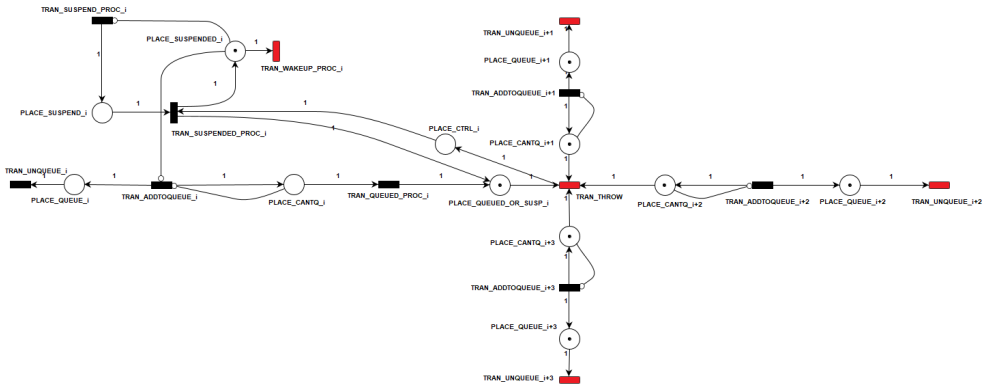


Figura 47: Secuencia de suspensión con nuevo `THROW` (4)

A.4 Suspensión y THROW con red simplificada

A.4.1 Problema

Con esta secuencia de disparos podemos ver cómo la solución propuesta en la iteración 4 no termina de solucionar el problema, ya que sólo funciona el equilibrado con THROW la primera vez. Vemos cómo funciona la secuencia que lleva a un núcleo a estado suspendido, cómo se sensibiliza THROW (en este caso tenemos un núcleo suspendido y uno encolando) y cómo esta transición permite a los núcleos no suspendidos volver a encolar.

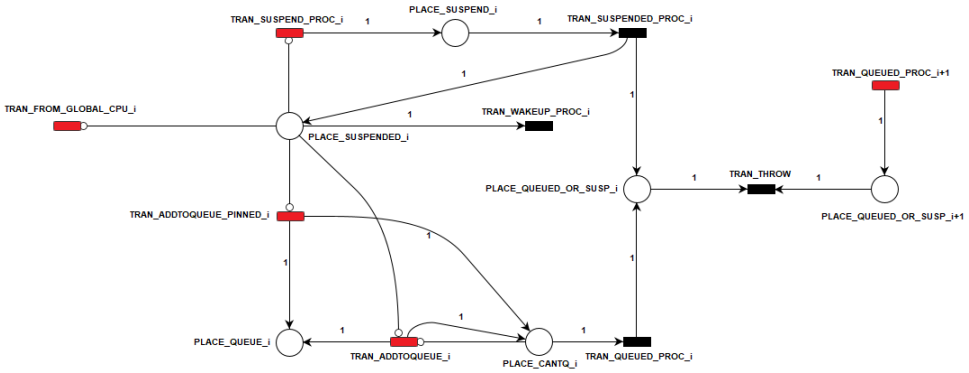


Figura 48: Problema suspensión y THROW (1)

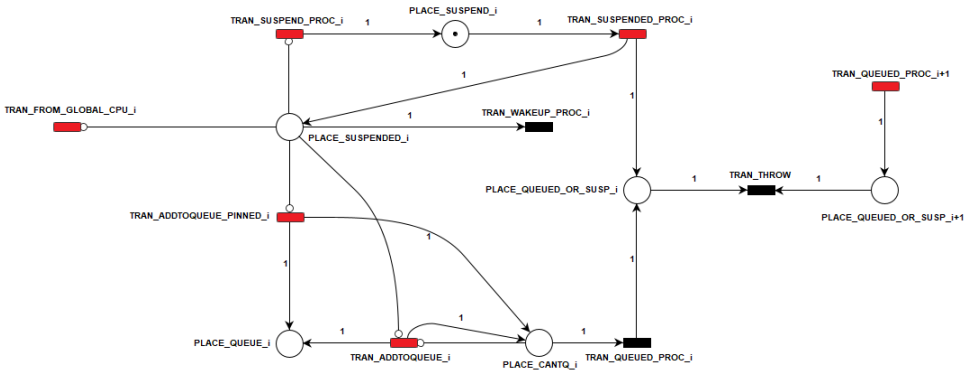


Figura 49: Problema suspensión y THROW (2)

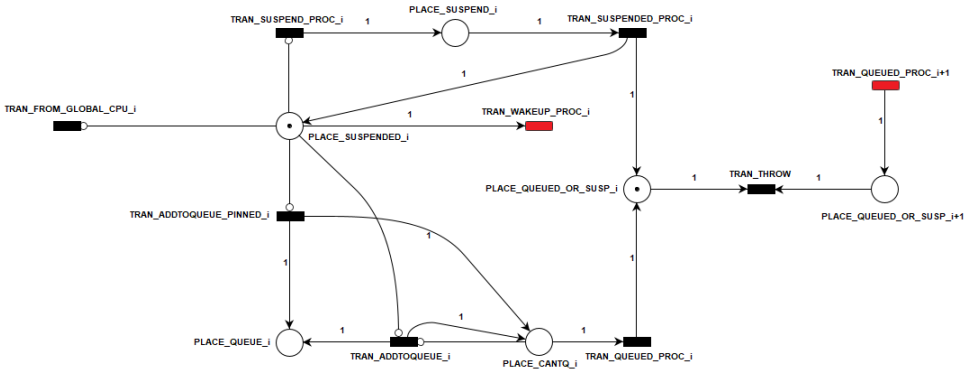


Figura 50: Problema suspensión y `THROW` (3)

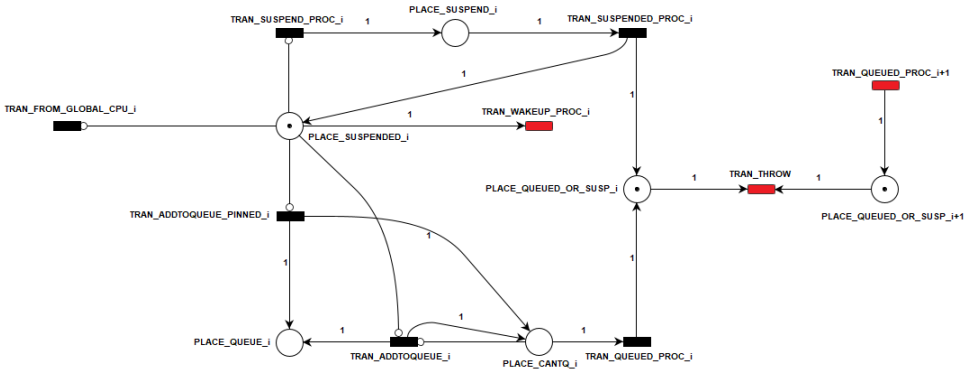


Figura 51: Problema suspensión y `THROW` (4)

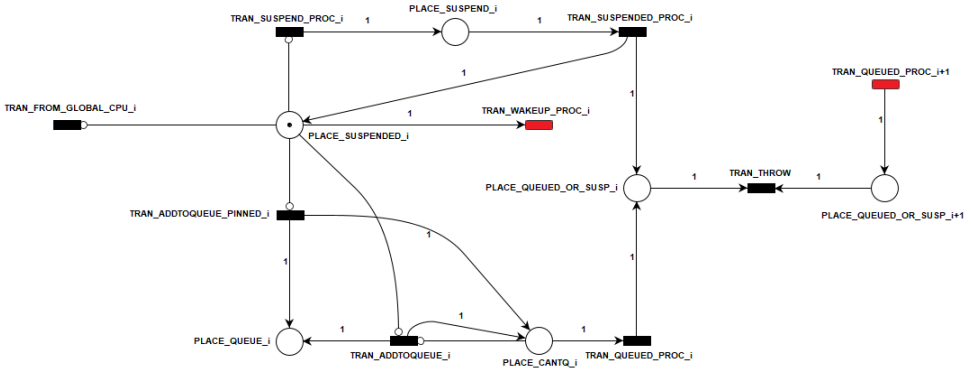


Figura 52: Problema suspensión y `THROW` (5)

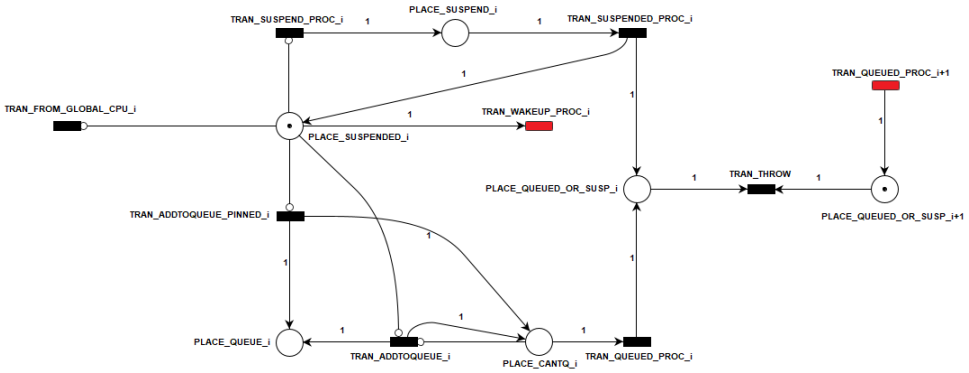


Figura 53: Problema suspensión y `THROW` (6)

A.4.2 Solución

La solución que se implementó fue que mediante código en el *kernel* se detecten los núcleos suspendidos cuando se dispara `THROW` y se llene la plaza `QUEUED_OR_SUSP` de los mismos, permitiendo así que `THROW` sea sensibilizada cuando los núcleos que pueden encolar, lo hagan. Así quedaría la red luego del disparo de `THROW`:

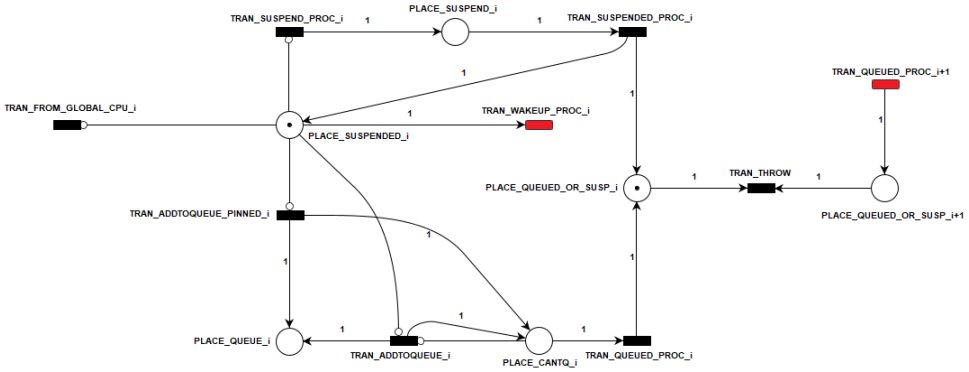


Figura 54: Solución por código y suspensión y `THROW`



B. Anexo: Configuración del entorno de trabajo resultante

B.1 Introducción

B.1.1 Vagrant

Vagrant es una herramienta desarrollada por HashiCorp. Esta nos permite rápidamente crear entornos de desarrollo virtualizados livianos, reproducibles y portables.

B.1.2 Ansible

Ansible es una herramienta de automatización para gestión de la configuración de equipos, despliegue de aplicaciones y aprovisionamiento de equipos.

B.2 Aprovisionamiento de VMs con el proyecto integrador

Usando las herramientas mencionadas anteriormente más software de virtualización como VirtualBox y KVM, podemos automatizar el proceso de obtención de una máquina virtual con lo necesario para testear y/o desarrollar sobre el proyecto.

Por defecto, las VMs se crearán con 4 cores y 4096 MB de memoria. Para cambiar esto se debe modificar en el archivo `vagrant/Vagrantfile.template` la sección `config.vm.provider`.

Ejecutando el *script* `setup.sh` pasando como argumento `freebsd<version_num>` (13, 14 ó 15) nos generará los archivos de configuración que Vagrant y Ansible necesitan para crear una nueva máquina virtual aprovisionada.



Esto se hará a partir de *templates* y sólo si se tienen instalados los paquetes necesarios en el sistema:

- `ansible`
- `vagrant`, con los siguientes plugins:
  - `vagrant-libvirt`
  - `vagrant-mutate`
  - `vagrant-timezone`
- `KVM`, `qemu` y `libvirtd` para las versiones FreeBSD 13 y 14
- `VirtualBox` para la versión FreeBSD 15 (`CURRENT`)

Luego, si se ejecuta el script `launch.sh`, Vagrant se encargará de crear la nueva máquina virtual y de encargarle a Ansible el aprovisionamiento de la misma. Esto es, se copian todos los archivos de código de *kernel* personalizados para el proyecto, los módulos de *kernel* creados y archivos de configuración para el sistema operativo. Por último, Ansible compilará este nuevo *kernel*, lo que puede demorar unos 20 minutos (dependiendo de las capacidades definidas) y reiniciará la máquina virtual para que lograr un *boot* con el nuevo *kernel* cargado.

Finalmente, para ingresar a la máquina virtual debemos seguir los comandos:

```
1 cd vagrant
2 vagrant ssh
```